Which Parts of a Tool's Documentation Does a Model Actually Need?

Randeep Singh
Lucio AI
randeep@lucioai.com

Abstract

Function-calling interfaces document each tool with four pieces of text: a name, a description, parameter names and parameter descriptions. We ask which of these a model actually depends on in stateful, multi-turn tasks. On the multi-turn tasks of the Berkeley Function Calling Leaderboard (BFCL), we ablate each component, then all four, for four models (GPT-6 Luna, Claude Haiku 5.5, Qwen3-30B-A3B and Gemma 4 26B-A4B), with paired tests against a measured noise floor. Only removing parameter descriptions significantly lowers task success for every model; removing tool names or tool descriptions stays within noise, though removing all four leaves almost nothing. Trajectories place the damage at argument construction rather than tool selection, and trace it to a handful of parameters whose descriptions were the only place an allowed value, a format or an entity type was stated. Writing those constraints into the schema as `enum`, `pattern` and clearer parameter names restores baseline success with every description left blank. On a second benchmark, Apple's ToolSandbox, where every component restates the others, no single one can be shown to matter. Models need each constraint stated once, in any carrier; on BFCL, parameter descriptions were simply the carrier that held them.

1 Introduction

Function calling is the standard way to connect a language model to software: the model is shown a list of tools, each a JSON Schema (Wright et al., 2022) with a name, a description and, per parameter, a name, a type and a description, and answers a user by emitting calls against that list (Anthropic, 2024; OpenAI, 2023). Practitioners rewrite tool descriptions and a line of work studies how to write them well (Guo et al., 2026; Hsieh et al., 2023; Qu et al., 2025a; Yuan et al., 2025), but there is no controlled evidence about which parts of that prose a model relies on inside a realistic task, with state to track, several turns to get through and a few dozen tools to choose from.

We take the 200 multi-turn base tasks of BFCL (Mao et al., 2024; Patil et al., 2025), hold everything fixed except the documentation shown to the model, and remove one of its four prose components at a time, then all four. Structural fields, tool outputs and backend state are kept, so the experiment measures the prose rather than what an agent can learn by interacting. Four models from four providers ran every task under every condition, three of them with reasoning on and off, with every comparison paired at the task level and a repeated baseline measuring noise. The single-component conditions were also run on Apple's ToolSandbox (Lu et al., 2024), which documents every tool redundantly.

Section 3 fixes the vocabulary: a tool call involves two decisions, *selecting* a tool and *constructing* its arguments, and a schema states what each decision needs through four *prose carriers* and a set of *structural fields*. The experiment ablates the carriers; the trajectories say which decision each carrier served. We find:

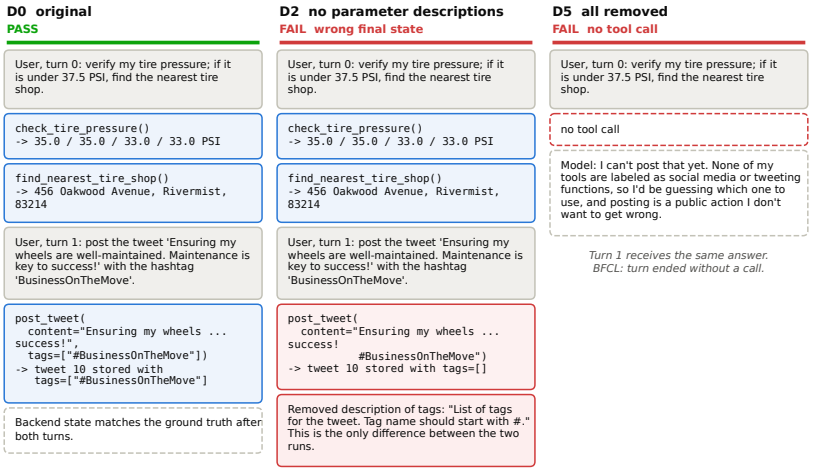


Figure 1: One task under three conditions (Claude Haiku 5.5, BFCL task 78). With the original schema (D0) the hashtag goes in the `tags` list. Without parameter descriptions (D2) the first turn is identical, but the hashtag is folded into the tweet body and the final state no longer matches. With names and descriptions removed (D5) the model declines to guess which tool posts. Appendix D gives the full trajectories.

- **Parameter descriptions are the only carrier every model depends on** (Section 5): removing them costs 14 to 33 percentage points, significant for every configuration. Removing tool names costs nothing, and removing tool descriptions costs 2.5 to 6 points, inside the measured noise.
- **The damage is at construction, not selection** (Section 6.1): the right tool is chosen as often as before, but the right tool with the right arguments falls by 11 to 16 points.
- **Six parameters carry the effect, each description the only carrier of a constraint** (Section 6.2): an allowed-value list, a format or an entity type no structural field stated. On the 91 tasks that use them the drop is 31 to 74 points; on the other 109 it is within noise.
- **Structure substitutes for the prose** (Section 6.3): writing those constraints into `enum`, `pattern` and two better parameter names, with every description blank, returns every model to baseline.
- **Where the semantics live does not matter, as long as they are somewhere** (Section 5): on ToolSandbox, whose carriers restate each other, removing any one costs nothing and removing all four costs 39 to 49 points.
- **Reasoning and exploration do not compensate** (Sections 5 and 6.4): the effect is the same size with reasoning on or off, models think longer without documentation and recover none of the loss, and no model recovers a task after an execution error once everything is removed.

2 Related work

Tool use has progressed from inserted API calls (Schick et al., 2023) and reasoning-and-acting prompting (Yao et al., 2023) to models trained on tens of thousands of APIs (Patil et al., 2023; Qin et al., 2024), with the schema interface of Section 3.1 now shared across providers (Anthropic, 2024; OpenAI, 2023). Qu et al. (2025b) decompose tool use into planning, selection, calling and response generation, the framing behind Section 3.3. API-Bank (Li et al., 2023) and BFCL's single-turn categories (Yan et al., 2024) score individual calls; BFCL V3 (Mao et al., 2024), τ-bench (Yao et al., 2025) and ToolSandbox (Lu et al., 2024) score a trajectory by its outcome,

which is what lets a documentation change be isolated (Section 3.2). Prior work on documentation treats it as one object to improve, by showing it can replace demonstrations (Hsieh et al., 2023) or by rewriting it (Guo et al., 2026; Qu et al., 2025a; Yuan et al., 2025); we ask which component carries the value. Clouâtre (2026) found, for two models, no measurable accuracy loss from moving parameter detail out of the tool description into per-parameter fields, the redundancy our D1 result shows. Robustness studies perturb the query, the toolkit, the tool interface or the tool outputs and find consistent degradation (Rabinovich and Anaby-Tavor, 2025; Xu et al., 2026; Zheng et al., 2026); Chattopadhyay et al. (2026) find that description and schema edits are less harmful than distractor tools with overlapping descriptions, matching our D1/D3 nulls. None removes a component outright while holding the benchmark fixed. On the form of the interface, Sigdel and Baral (2026) find that schema conditions reduce interface misuse but not semantic misuse, the design closest to our D6; Johnson et al. (2025) find that replacing JSON tool calls with natural-language tool selection raises accuracy across ten models, a step our ablations barely touch; and Ray (2026) show that *enforcing* a schema during decoding can keep outputs valid while lowering executable accuracy.

3 Background

3.1 The function-calling interface

Provider-native function calling (OpenAI, 2023) and the Model Context Protocol (Anthropic, 2024) share one format: each tool is a JSON object with a `name`, a `description` and a `parameters` JSON Schema (Wright et al., 2022), and every model here receives exactly that serialised schema (BFCL's "FC mode" (Patil et al., 2025)). Formally, a tool is a tuple $\tau = (n, d, P)$ of a name, a description and a parameter specification. Each $p \in P$ has a name, a type, a description and *structural fields*, the ones JSON Schema defines with a fixed meaning: `required`, `default`, an `enum` of admissible values, a `pattern` a string must match, and nested specifications for object-typed parameters. The model is shown a tool set $\mathcal{T} = \{\tau_1, \ldots, \tau_K\}$ and answers with text or with calls (n, a), where a should conform to P.

The schema thus mixes two kinds of information. The structural fields are machine-readable and have one meaning. The *prose carriers* are four: the tool name, the tool description, the parameter names and the parameter descriptions. A description may say anything, including what a structural field could have said: "Type of order (Buy/Sell)" states an admissible-value set and "Tags should start with #" a format, in a place no parser enforces. We call a constraint on an argument value *structural* if a structural field expresses it and *prose-only* if only a description states it.

3.2 The interaction loop and its evaluation

A tool-calling conversation alternates between a user and the model. Within a user *turn* the model takes one or more *steps*: it is sent the conversation so far and the tool set, and replies with calls or with text. Each call is executed against a backend, whose result or error is appended before the next step; a text reply ends the turn. A *stateful* backend, such as a file system, lets a call change what later calls and turns see, so a wrong call produces an observable error or result that the model has further steps to react to.

Single-call benchmarks (Li et al., 2023; Yan et al., 2024) compare each call with a reference. Stateful benchmarks compare outcomes: BFCL's multi-turn categories (Mao et al., 2024) replay a ground-truth call sequence on a copy of the backends and pass a turn only if the final states agree and any required outputs appear; τ-bench (Yao et al., 2025) and ToolSandbox (Lu et al., 2024) simulate the user with a language model, and ToolSandbox scores a trajectory by similarity to milestones. Outcome scoring is what isolates a documentation ablation: the model may reach

the required state by any sequence of calls, while backend, user turns and ground truth are identical in every condition.

3.3 Two decisions and what they need

At every step the model that acts makes two decisions: it *selects* a tool $n \in \mathcal{T}$ and *constructs* an argument object a for it. Selection needs to know what each tool *does*. Construction needs to know what each parameter *is*, which are required and which values are admissible, the constraints of Section 3.1, plus the values themselves, which come from the user's request, earlier results or convention.

Each piece of that information may be stated in several carriers at once: a tool's purpose in its name, its description and often its parameter list; an admissible set in an `enum` or in a description. Where two carriers state the same thing, removing one leaves the other. Where one carrier is the only place something is stated, removing it removes the information, and the model must infer it, discover it by interacting (*interactive recovery*), or fail. So removing a carrier should cost nothing if its content is redundant, and should cost at the decision that needs the content if it is not. In the trajectories, a *selection failure* shows as the right tool never being called, a *construction failure* as the right tool called with wrong values, and recovery as a later call that corrects an earlier one.

4 Experimental setup

4.1 Benchmarks

BFCL multi-turn base. The `multi_turn_base` category (Mao et al., 2024) has 200 tasks of one to seven user turns (median four) over eight stateful Python backends (file system, vehicle control, trading, travel, messaging, a Twitter-like API, ticketing, math). A task involves one or two backends, and the model is shown every function of those backends, 18 to 39 tools (median 28). A turn passes if the backend state matches the state the ground-truth calls produce and any required outputs were produced; a task passes only if every turn passes; a turn may take at most 20 steps. We used BFCL at commit `6ea57973` (Gorilla LLM contributors, 2026) at its default temperature of 0.001, unmodified.

ToolSandbox. The single-component conditions were also run on ToolSandbox (Lu et al., 2024) (commit `c8571d7`), which differs in every respect that could carry a documentation effect: the user is simulated by a language model (GPT-4o-2024-05-13, as in the original paper); the tools are Python functions over a phone-like world whose schemas are generated from docstrings, so every description is one short line; and the score is a similarity in $[0, 1]$ to the scenario's milestones rather than a verdict. We run the 91 base scenarios that need no paid web API, each with 24 to 28 tools, for the four models in their default reasoning setting (Appendix C).

4.2 Documentation conditions

Each condition is an operation on the tuple (n, d, P) that leaves the structural fields untouched (Table 1). D1 to D4 each remove one carrier, so each measures what that carrier contributed beyond the other three, the user's request and the tool outputs. D5 removes all four and tests *reduced* semantics rather than none: types, requiredness, enums, outputs and the request remain, and tools stay distinguishable by argument structure. With generic names the model's calls are translated back before BFCL executes and scores them, so backend and evaluator see what they would in D0.

D6 and D7 test the explanation Section 6.2 arrives at. Both blank every parameter description as D2 does. D6 then makes every prose-only constraint of the BFCL schemas structural: each

Table 1: Documentation conditions. Generic tool names are `tool_01`...`tool_NN` in listing order; generic parameter names are `arg_1`...`arg_n` per tool, recursively. Structural fields are kept throughout. †D1 also blanks the response-schema descriptions. ‡D6 renames `cityA`/`cityB` to `zipcodeA`/`zipcodeB`.

	Tool names	Tool descriptions	Parameter names	Parameter descriptions
D0	original	original	original	original
D1	original	**removed**†	original	original
D2	original	original	original	**removed**
D3	**generic**	original	original	original
D4	original	original	**generic**	original
D5	**generic**	**removed**	**generic**	**removed**
D6	original	original	original‡	**removed**; constraints in schema
D7	original	original	original	**removed**; six kept (Table 3)

parameter whose description holds an allowed-value list receives a JSON Schema `enum` (11 parameters across three backends), `post_tweet.tags` and `mentions` receive item patterns `^#` and `^@`, and `estimate_distance.cityA`/`cityB` are renamed `zipcodeA`/`zipcodeB`. D7 instead keeps only the six descriptions of Table 3.

4.3 Models and harness

We evaluated GPT-6 Luna (OpenAI, 2026), Claude Haiku 5.5 (Anthropic, 2026), Qwen3-30B-A3B-Instruct-2507 (Yang et al., 2025) and Gemma 4 26B-A4B (Google DeepMind, 2026) through OpenRouter (OpenRouter, 2026), all in native function-calling mode. Luna and Haiku reason by default and were also run with reasoning off; Gemma 4 does not and was also run with it on; Qwen3 has no switch. This gives seven configurations, each run on all 200 tasks under D0 to D5, with D6 and D7 in each model's default setting. Comparisons across models confound provider, architecture and reasoning; the within-configuration comparisons that carry the claims do not. A second Claude Haiku 5.5 run at D0, eight hours after the first, measures run-to-run noise. Qwen3's requests were split between two providers in 336 of its 1 200 trajectories, with lower success on those in every condition; the D2 effect is present on both subsets.

The harness subclasses BFCL's OpenAI-compatible handler and differs from a stock run only in routing to OpenRouter, applying the condition, translating generic names back, and recording one file per trajectory with the exact tools sent, every raw request and response, BFCL's per-turn state and the verdict (10 200 BFCL trajectories and 2 184 ToolSandbox plays). Four Qwen3 trajectories ended in a provider error and count as failures.

4.4 Metrics and statistics

Task success is BFCL's own verdict. The behavioural metrics are statistics of the loop of Section 3.2, computed from the trajectories without model-based judging. A *call* is a tool call BFCL decoded and a *step* one model request; a call is *on-path* if its tool appears in the ground truth, *off-path* if not, and *failed* if the backend returned an error.

The two decisions are measured against the ground truth. For each ground-truth call in turn t, a *tool hit* means the model called that tool in any step of turns 0 to t, and an *exact hit* means one of those calls also had identical arguments (no case folding, so `"buy"` does not match `"Buy"`). *Tool-selection accuracy* is tool hits over ground-truth calls and *argument accuracy given the right tool* is exact hits over tool hits; both are conservative in every condition equally. For each failed task the *first divergence* is the earliest ground-truth call without an exact hit, a construction failure if the tool was called with other arguments and a selection failure if it was never called. When the right tool was called with wrong arguments, the parameters that differ from the closest same-tool call are recorded, and the D2 count minus the D0 count isolates the

Table 2: Task success (%) per condition, 200 tasks per cell, with the paired change versus D0 in percentage points. Bold, starred cells are significant at the Bonferroni threshold $p < 0.0014$ (exact McNemar test; discordant cells, intervals and p-values in Table 4).

Model	D0	D1	D2	D3	D4	D5
GPT-6 Luna, reasoning	56.0	52.5 (−3.5)	**32.5** (−23.5)*	57.5 (+1.5)	58.5 (+2.5)	**1.0** (−55.0)*
GPT-6 Luna, no reasoning	54.0	48.5 (−5.5)	**32.0** (−22.0)*	59.5 (+5.5)	56.0 (+2.0)	**0.0** (−54.0)*
Claude Haiku 5.5, reasoning	50.5	48.0 (−2.5)	**36.5** (−14.0)*	55.5 (+5.0)	**61.5** (+11.0)*	**0.0** (−50.5)*
Claude Haiku 5.5, no reasoning	47.0	42.0 (−5.0)	**27.5** (−19.5)*	47.5 (+0.5)	46.5 (−0.5)	**0.0** (−47.0)*
Qwen3-30B-A3B, no reasoning	54.5	48.5 (−6.0)	**28.5** (−26.0)*	52.5 (−2.0)	**43.0** (−11.5)*	**0.0** (−54.5)*
Gemma 4 26B-A4B, no reasoning	72.0	69.0 (−3.0)	**39.0** (−33.0)*	71.5 (−0.5)	74.5 (+2.5)	**0.0** (−72.0)*
Gemma 4 26B-A4B, reasoning	76.0	70.5 (−5.5)	**42.5** (−33.5)*	78.0 (+2.0)	76.0 (+0.0)	**2.5** (−73.5)*

errors that removing a description produced. *Recovery rate* is the share of tasks with at least one failed call that still passed.

Every task appears in every condition, so each ablation is compared with D0 by an exact two-sided McNemar test (McNemar, 1947) on the discordant cells, with a 95% interval for the paired difference from a task-level bootstrap of 2 000 resamples (Efron, 1979). With 35 comparisons (five ablations × seven configurations) the Bonferroni threshold is $p < 0.0014$ (Dunn, 1961).

5 Results

Removing parameter descriptions is the only single ablation that hurts every model. Every configuration in Table 2 (Figure 5 shows the same changes as a heatmap) has the same shape: a small dip at D1, a large drop at D2, no change at D3 and a collapse at D5. D2 and D5 are significant for all seven configurations at the corrected threshold, and the D2 discordant cells are lopsided: tasks passing only in D0 outnumber tasks passing only in D2 by 42 to 75 against 5 to 16 (Table 4). D1 and D3 have discordant cells of similar size in both directions and every interval includes zero, although all seven D1 differences are negative (−2.5 to −6.0 pp). D4 is harmless for Luna and Gemma 4, costs Qwen3 11.5 pp ($p = 0.001$), and shows an apparent +11.0 pp gain for Haiku with reasoning that is absent with reasoning off.

Two identical runs differ by about as much as D1 or D3. Claude Haiku 5.5's second D0 run passed 56.5% of tasks against 50.5% in the first, with 9 tasks passing only in the first run and 21 only in the second ($\Delta = +6.0$ pp, $p = 0.0428$; Table 5). The backends are deterministic, so every discordance is the model's, and this gap is the floor against which single-condition differences must be read. D3 and, for three models, D4 sit at it and are null results. D1 is a possible small cost: Haiku's D1 gap widens to −8.5 pp ($p = 0.008$) against the second run and D1 is negative for all seven configurations, but one run per condition cannot resolve a few points. Haiku's D4 gain shrinks to +5.0 pp ($p = 0.13$) against the second run and we treat it as noise. D2 and D5 are an order of magnitude above the floor and significant against either run.

Reasoning changes the level, not the effect. The three pairs that differ only in the reasoning switch show D2 drops of −23.5 and −22.0 pp (Luna on and off), −14.0 and −19.5 pp (Haiku) and −33.0 and −33.5 pp (Gemma 4). Reasoning budgets respond to the interface even though success does not (Table 6): per task, Luna spends 330 reasoning tokens at D0, 390 at D2 and 2 294 at D5, and Gemma 4 with reasoning on 1 110, 2 554 and 8 659. Nor do the models remember the benchmark: no D3, D4 or D5 trajectory contains an original tool or parameter name (Appendix A).

Where the semantics live does not matter. ToolSandbox is the fully redundant case of Section 3.3: its tool names are full phrases (`set_wifi_status`), its descriptions paraphrase

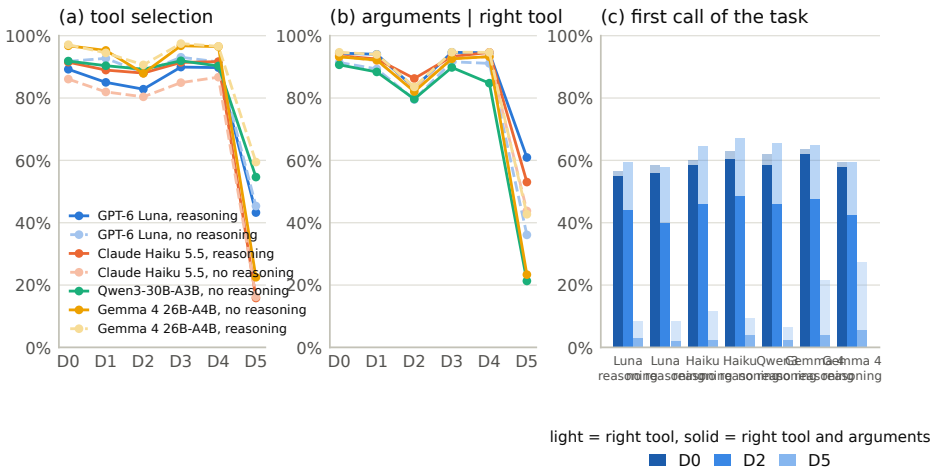


Figure 2: (a) Tool-selection accuracy. (b) Argument accuracy given the right tool. (c) First call of the task: share of tasks whose first call used a ground-truth tool (light) and used it with the ground-truth arguments (solid), for D0, D2 and D5. Numbers in Table 8.

the name, its parameter names paraphrase their descriptions, and none of its 72 parameter descriptions carries a constraint the schema lacks. The prediction is that no single carrier can be shown to matter, and none can: D1 to D4 each leave every model within a few points of its baseline similarity (D2 moves it by -6.3 to $+2.0$, significant for no model), and the ablated run still calls every tool the baseline used in 66 to 91% of the scenarios the baseline solved. Removing all four costs every model 39 to 49 points ($p < 0.0001$) and that share falls to 0 to 13% (Appendix C). The models depend on a tool's semantics being stated in some carrier, not in any particular one.

6 Analysis

Section 3.3 says where parameter descriptions could matter: at selection, at construction, or in interactive recovery. The trajectories tell the three apart.

6.1 Selection versus construction

D2 leaves selection intact and damages construction. Between D0 and D2 tool-selection accuracy moves by 2.6 to 8.8 points, part of which is cascade, since a turn derailed by a wrong argument may never reach its later tools; argument accuracy given the right tool falls for every configuration, from 90.7–94.7% to 79.5–86.3% (Figure 2a,b). The first call of each task is free of cascade (Figure 2c): in D2 the right tool is chosen first at least as often as in D0 for every configuration, but the right tool with the right arguments falls by 11.0 to 16.0 points. The models know what to call; they no longer know what to put in it. The first divergence of failed tasks agrees (Figure 6): relative to D0, construction failures rise by 34 to 66 per configuration and selection failures by 2 to 11. D5 is different in kind: tool-selection accuracy collapses to 16 to 60% and most failures begin with a tool never called, as Section 3.3 predicts once every carrier of a tool's purpose is gone.

6.2 Which descriptions matter

Six parameters carry the effect, and each description was the only carrier of a constraint. Table 3 links each new argument error to the description that was removed. The same six parameters lead for every configuration, and each description held a prose-only constraint

Table 3: Parameters whose values go wrong when their description is removed. For every ground-truth call matched by the right tool but wrong arguments, the differing parameters were recorded. Counts are pooled over the seven configurations (1 400 task runs per condition), sorted by D2−D0; D2 counts per configuration on the right (−: reasoning off, +: on). The second column quotes the description D2 removed.

Tool.parameter	Removed description	D2	D0	Luna	Luna$^-$	Haiku	Haiku$^-$	Qwen3	Gemma	Gemma$^+$
`place_order.order_type`	*Type of the order (Buy/Sell).*	**186**	0	26	28	27	22	27	28	28
`startEngine.ignitionMode`	*The ignition mode of the vehicle. [Enum]: ["START", "STOP"]*	**174**	0	25	22	14	26	28	30	29
`estimate_distance.cityA`	*The zipcode of the first city.*	**134**	7	20	20	20	20	16	19	19
`estimate_distance.cityB`	*The zipcode of the second city.*	**134**	7	20	20	20	20	16	19	19
`post_tweet.tags`	*List of tags for the tweet. Tag name should start with #. This is only relevant if the user wants to add tags to the tweet.*	**130**	4	22	24	18	15	12	20	19
`post_tweet.mentions`	*List of users mentioned in the tweet. Mention name should start with @. This is only relevant if the user wants to add mentions to the tweet.*	**75**	5	15	14	11	6	8	11	10
`get_available_stocks.sector`	*The sector to retrieve stocks from (e.g., 'Technology').*	**17**	0	3	4	0	2	4	1	3
`post_tweet.content`	*Content of the tweet.*	**62**	47	6	5	17	17	3	11	3
`lockDoors.door`	*The list of doors to lock or unlock. [Enum]: ["driver", "passenger", "rear_left", "rear_right"]*	**22**	7	2	7	1	1	4	4	3
`get_zipcode_based_on_city.city`	*The name of the city.*	**14**	0	1	1	4	1	4	1	2

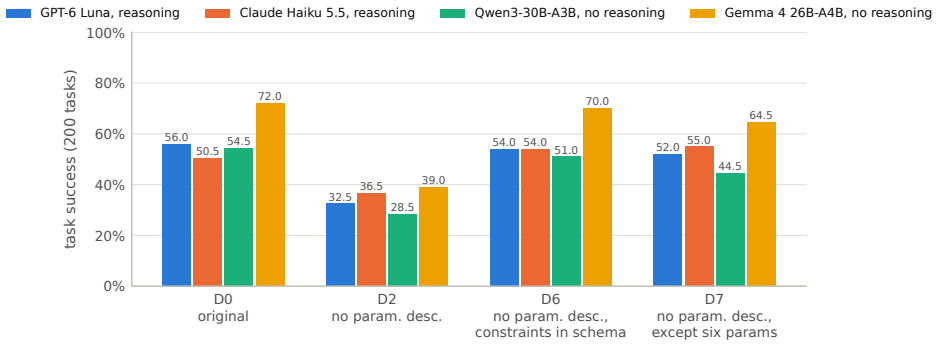


Figure 3: D6 and D7, each model in its default reasoning setting. Both blank every parameter description, as D2 does. D6 moves the prose-only constraints into the schema; D7 keeps only the six descriptions of Table 3. Paired tests in Table 11.

in the sense of Section 3.1: an allowed-value list (`order_type`: "Buy/Sell"; `ignitionMode`: "START, STOP"), an entity type the name hides (`cityA`/`cityB` take zip codes), or a format (`tags` start with `#`, `mentions` with `@`). Without it, models wrote `"buy"` for `"Buy"`, passed city names for zip codes, or folded the hashtag into the tweet body (the `content` row mirrors `tags`). `place_order` and `post_tweet` store the wrong value silently, so only the final-state check catches it; `startEngine` and `estimate_distance` return an explicit error, which the model usually did not act on (Section 6.4).

At task level, the D2 effect is an effect on the tasks that use them. The ground truth of 91 of the 200 tasks uses at least one of the six. Of the tasks that pass in D0 and fail in D2, 29 of 46 to 68 of 75 are among those 91, and the D2 drop on the 91 tasks is 30.8 to 73.6 pp while on the other 109 it is between −10.1 and +4.6 pp, within the noise floor for six configurations (Table 10). Parameter descriptions matter here less as meaning than as the place where this benchmark keeps its enums, formats and entity conventions, and Section 3.3 predicts that moving a constraint from one carrier to another should change nothing.

6.3 Restoring the structure without the prose

Writing the constraints into the schema restores baseline with every description blank. If the descriptions that matter are the sole carriers of constraints the schema could express, D6 should perform like D0 and D7 should recover most but not all of the loss. Both hold (Figure 3, Table 11). D6 reaches 54.0, 54.0, 51.0 and 70.0% (Luna, Haiku, Qwen3, Gemma 4) against 56.0, 50.5, 54.5 and 72.0% in D0: every model is within 3.5 points of baseline, no D6–D0 difference is significant, and every D6–D2 difference is, recovering 17.5 to 31.0 points. D7 reaches 52.0, 55.0, 44.5 and 64.5%, most of the D2 loss but 4 to 10 points below D0 for three models, and the parameters that go wrong in D7 but not in D6 are the remaining prose-only enums that only D6 converts, led by `lockDoors.door`.

The constraint was read, not merely enforced. A provider may enforce an `enum` during decoding. Counting every value emitted for the 13 constrained parameters, the share outside the constraint is 37 to 69% in D2 and at most 0.7% in D6 for every model; Qwen3's three off-constraint values show its providers did not enforce the enum, and the other three models were already at 0.0% in D0, where the constraint was prose no decoder could enforce. Enforcement is not needed to explain the recovery, which matters because it is not free (Ray, 2026). D6 does rely on the serving stack passing `enum` and `pattern` through, which OpenRouter did here; BFCL itself strips several constraint fields for Gemini-style endpoints.

6.4 Behaviour, recovery and failure outcomes

The third possibility of Section 3.3 is that a model compensates by interacting. Under the single ablations it does not: D2 adds 0.1 to 2.8 calls per task over D0, and the composition shifts rather than the count, with failed calls per task rising from at most 0.03 to 0.10–0.51 (Appendix B). Recovery after an execution error is poor in D2, 5 to 22% of the 19 to 30 tasks that contained one, and all but absent in D5, where six configurations recovered none of their 17 to 102 such tasks. D5 produces two opposite behaviours, neither of which passes tasks: Haiku and Luna with reasoning end the first turn with no call in 22 to 35% of tasks, while Qwen3 and Gemma 4 with reasoning probe to the step cap. Four case studies selected by a fixed rule, one per model, illustrate these mechanisms (Appendix D); three fail in D2 by the hashtag-in-content error of Figure 1, the fourth is a D0/D2 discordance with no documentation cause.

7 Discussion

For tool designers. The descriptions whose removal hurt were not explaining what a parameter means; no model needs prose to learn that `order_type` is the type of the order. They carried an allowed-value list, a format rule or an entity convention that JSON Schema could have expressed as `enum`, `pattern` or a better name. This suggests a simple audit: for every parameter description, ask whether it states a constraint the structural fields do not, and if so move it there, keeping the prose for what structure cannot say. Where the serving stack strips structural fields, the prose is the only channel and D2 is the relevant condition. D4 points the same way: for three of four models, replacing a parameter name with `arg_N` while keeping its description cost nothing; Qwen3's −11.5 pp shows the dependence on names is model-specific.

Why tool names and descriptions were free. BFCL's tools have descriptive names, the user prompts name the actions in plain language, and the outputs are fully informative, so the description is redundant with the name and the name with the parameter list; D5 shows the information was used, from more than one place. ToolSandbox, where fourteen tools take at most one argument, shows a nominal 6-point D3 loss for Gemma 4, a first sign of the name mattering

once argument structure is less distinctive; with semantically overlapping tools (Chattopadhyay et al., 2026; Rabinovich and Anaby-Tavor, 2025) D1 and D3 should matter more.

Interaction does not substitute. Declining to act fails because the evaluator requires the state change; probing fails because BFCL caps a turn at 20 steps and scores the final state. Recovery is poor even in D2, which suggests that models treat an execution error as a signal to try another tool or give up rather than to re-read the schema; whether dry runs, more steps or structured diagnostics (Sigdel and Baral, 2026) would help is open.

For benchmark designers. The D2 effect flows through a dozen parameters in three backends and disappears when their constraints are written into the schema; a benchmark that encoded its enums and formats structurally would isolate task understanding from prose parsing. Conversely, frontier models failing on `"buy"` versus `"Buy"` shows how literally stateful benchmarks score: BFCL's single-turn checker normalises case and punctuation, but its multi-turn checker compares backend attributes with Python equality (Appendix A), so a value the one accepts fails the other. How much of a reported function-calling score rests on such conventions is worth knowing.

8 Limitations

One run per condition. Two identical runs disagree on tens of tasks (Section 5), so differences of a few points, including the D1 cost and Haiku's D4 gain, cannot be resolved by this design; the D2, D5 and D6-versus-D2 effects are an order of magnitude larger. ToolSandbox was run once, with a simulated user whose variation enters every difference.

D5 is reduced semantics, not none. Types, requiredness, enums, outputs and the user's request remain, and the collapse is partly a property of an evaluator that caps steps and scores final state.

The metrics are strict, and D6 is a targeted test. Exact hits treat equivalent but differently spelled values as misses, equally in every condition, and the Qwen3 case study shows that a single discordant task need not be a documentation effect. D6 converts the constraints that fail in D2 here and depends on providers passing `enum` and `pattern` through; it does not say how a model fares on an API whose constraints were never written anywhere.

Provider and reasoning settings were not fully controlled. Routing was left to OpenRouter, a third of Qwen3's trajectories touched a second provider, and the reasoning switch sets no effort level.

9 Conclusion

A function-calling schema states what a model needs through several carriers at once. With everything but the documentation held fixed, four models from four providers depend on the same carrier, parameter descriptions, at the same decision, argument construction, for the same six parameters. What those descriptions supplied was not meaning but constraints: written into the schema as enums, patterns and precise names, they restore baseline with every description blank. Tool names and descriptions were free to remove, reasoning and exploration did not compensate, and on ToolSandbox, where every carrier restates the others, none could be shown to matter alone. The models need each constraint stated once, in any carrier: tool designers should put structure in structural fields, and benchmark designers should know that a score can rest on documentation conventions unrelated to the task.

Reproducibility statement

The harness, the condition definitions (including the exact D6 transformation), the analysis code and the paper sources are at https://github.com/Eazydraft/exp-tool-semantics-ablation, with BFCL and ToolSandbox as pinned submodules. Every number, table and figure is computed from the 12 384 trajectory files under `runs/main/trajectories/`, each holding the exact tool schemas sent, every raw request and response, BFCL's per-turn state and the verdict; `./browse.sh` shows any trajectory with the schemas as the model saw them.

Acknowledgements

This study used the BFCL harness, backends and evaluator (Gorilla LLM contributors, 2026; Patil et al., 2025) and ToolSandbox (Lu et al., 2024) unchanged; we thank their authors. Models were accessed through OpenRouter (OpenRouter, 2026).

References

Anthropic. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol, November 2024.

Anthropic. Introducing Claude Haiku 5.5. https://www.anthropic.com/claude-haiku-5-5, October 2026. Announced 2026-10-07. System card: https://www.anthropic.com/claude-haiku-5-5-system-card.

Aniruddha Chattopadhyay, Kanishka Halder, and Partha Sarathi Roy. Robust function calling for LLM agents under interface drift, 2026. URL https://arxiv.org/abs/2610.23342.

Hugues Clouâtre. MCP parameter-description ablation: Token savings without accuracy loss. Zenodo, version 1.0.0, September 2026. Pre-registered experiment; code at https://github.com/clouatre-labs/param-description-experiments.

Olive Jean Dunn. Multiple comparisons among means. *Journal of the American Statistical Association*, 56(293):52–64, 1961. doi: 10.1080/01621459.1961.10482090.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7 (1):1–26, 1979. doi: 10.1214/aos/1176344552.

Google DeepMind. Gemma 4: Byte for byte, the most capable open models. https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/, April 2026. Announced 2026-04-02. Model card: https://huggingface.co/google/gemma-4-26b-a4b-it.

Gorilla LLM contributors. Gorilla: Berkeley function calling leaderboard source code. GitHub repository https://github.com/ShishirPatil/gorilla, 2026. Commit `6ea57973`, 2026-03-23, directory `berkeley-function-call-leaderboard`.

Ruocheng Guo, Kaiwen Dong, Xiang Gao, and Kamalika Das. Learning to rewrite tool descriptions for reliable LLM-agent tool use, 2026. URL https://arxiv.org/abs/2602.20426.

Cheng-Yu Hsieh, Si-An Chen, Chun-Liang Li, Yasuhisa Fujii, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. Tool documentation enables zero-shot tool-usage with large language models, 2023. URL https://arxiv.org/abs/2308.00675.

Reid T. Johnson, Michelle D. Pain, and Jordan D. West. Natural language tools: A natural language approach to tool calling in large language agents, 2025. URL https://arxiv.org/abs/2510.14453.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3102–3116. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.187.

Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities, 2024. URL https://arxiv.org/abs/2408.04682.

Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Jason Huang, Vishnu Suresh, Yixin Huang, Xiaowen Yu, Joseph E. Gonzalez, and Shishir G. Patil. BFCL V3: Multi-turn & multi-step function calling. Gorilla blog, UC Berkeley, September 2024. URL https://gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html. Released 2024-09-19, last updated 2024-12-10.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

OpenAI. Function calling and other API updates. https://openai.com/index/function-calling-and-other-api-updates/, June 2023.

OpenAI. Introducing GPT-6 Sol and Luna. https://openai.com/index/introducing-gpt-6-sol-and-luna/, September 2026.

OpenRouter. OpenRouter: A unified interface for LLMs. https://openrouter.ai, 2026. Accessed 2026-11-02.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs, 2023. URL https://arxiv.org/abs/2305.15334.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.16789.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. From exploration to mastery: Enabling LLMs to master tools via self-driven interactions. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025a. URL https://arxiv.org/abs/2410.08197. arXiv:2410.08197.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Tool learning with large language models: A survey. *Frontiers of Computer Science*, 2025b. doi: 10.1007/s11704-024-40678-2. URL https://arxiv.org/abs/2405.17935. arXiv:2405.17935.

Ella Rabinovich and Ateret Anaby-Tavor. On the robustness of agentic function calling. In *Proceedings of the 5th Workshop on Trustworthy NLP (TrustNLP 2025)*, pages 298–304. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.trustnlp-main.20. arXiv:2504.00914.

Jaideep Ray. The constraint tax: Measuring validity-correctness tradeoffs in structured outputs for small language models, 2026. URL https://arxiv.org/abs/2605.26128.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. arXiv:2302.04761.

Akshey Sigdel and Rista Baral. Schema first tool APIs for LLM agents: A controlled study of tool misuse, recovery, and budgeted performance, 2026. URL https://arxiv.org/abs/2603.13404.

Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. JSON Schema: A media type for describing JSON documents, draft 2020-12. Internet-Draft, https://json-schema.org/draft/2020-12/json-schema-core, 2022.

Yuanjian Xu, Jinyi Han, Jianing Hao, Yuxuan Sun, Fanyang Lu, Richeng Xuan, Guang Zhang, and Zhichao Hu. AgentPerturb: Evaluating LLM agent robustness under controlled interaction perturbations, 2026. URL https://arxiv.org/abs/2610.21624.

Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Ion Stoica, Joseph E. Gonzalez, Tianjun Zhang, and Shishir G. Patil. Berkeley function calling leaderboard. Gorilla blog, UC Berkeley, 2024. URL https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html. Last updated 2024-08-19.

An Yang et al. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388. Model card for the July 2025 instruct update: https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct-2507.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2406.12045. arXiv:2406.12045.

Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Yongliang Shen, Kan Ren, Dongsheng Li, and Deqing Yang. EasyTool: Enhancing LLM-based agents with concise tool instruction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 951–972. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.naacl-long.44. arXiv:2401.06201.

YiShan Zheng, Yuan Wu, and Yi Chang. ToolRobustBench: Stage-wise perturbation evaluation and failure diagnosis for tool-calling agents, 2026. URL https://arxiv.org/abs/2608.23635.

APPENDIX

A Additional results on BFCL

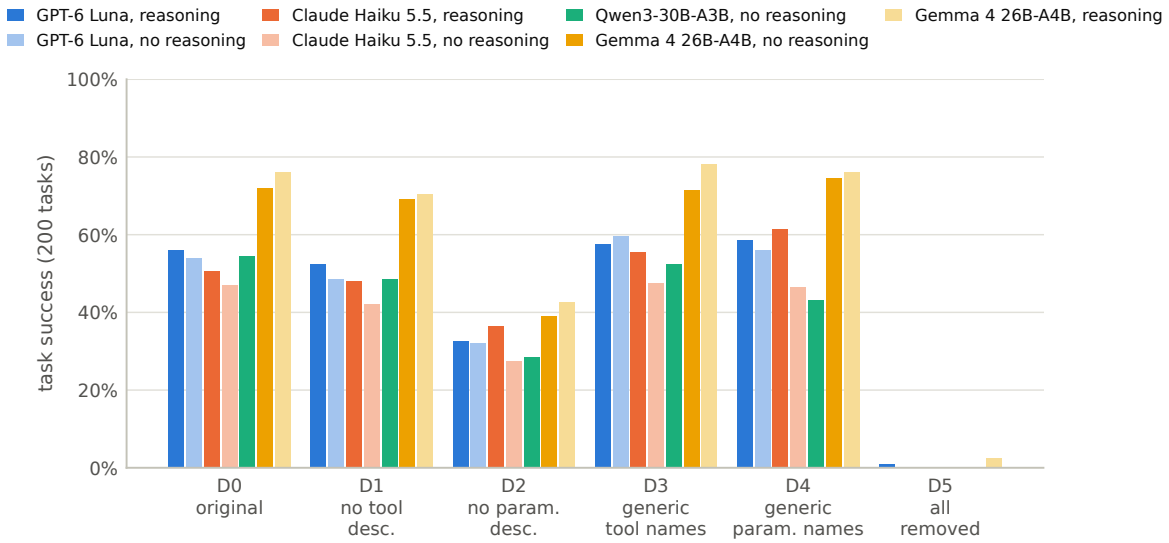


Figure 4: Task success rate by condition for the seven configurations (lighter tint: the non-default reasoning setting). Each bar is 200 tasks; bars within a configuration are paired.

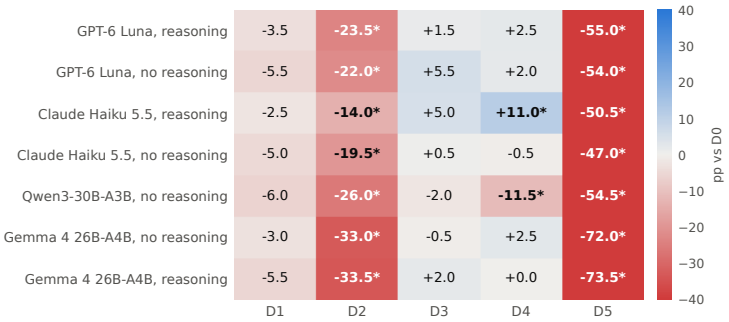


Figure 5: Change in task success relative to D0, in percentage points. Starred, bold cells are significant at $p < 0.0014$ (exact McNemar test, Table 4).

Memorisation check. Across the 1 515 to 3 927 tool calls each default configuration made in D3 and D5, and the 3 886 to 7 055 calls each reasoning variant made in D3, D4 and D5, no configuration emitted a single original tool name, and across D4 and D5 none emitted a single original parameter name. The only non-generic strings were eight malformed generic names from Qwen3 (`tool_6` for `tool_06`) and ten non-generic argument keys from Luna with reasoning.

Checker strictness. At the pinned commit, `standardize_string` in `bfcl_eval/eval_checker/ast_eval/ast_checker.py` removes spaces and punctuation and lower-cases both sides before the single-turn comparison, whereas `_compare_instances` in `bfcl_eval/eval_checker/multi_turn_eval/multi_turn_checker.py` tests every public attribute of the model's backend instance against the ground-truth instance with Python equality, and the multi-turn response checker matches serialised outputs without normalisation.

Table 4: Each ablation versus D0 on the same 200 tasks: the two discordant cells, the paired difference Δ in percentage points with its 95% task-level bootstrap interval (2 000 resamples), the exact two-sided McNemar p, and the Bonferroni verdict for 35 comparisons ($p < 0.0014$).

Model	Cond.	both pass	D0 only	abl. only	both fail	Δ pp	95% CI	McNemar p	$p < 0.0014$
GPT-6 Luna, reasoning	D1	88	24	17	71	-3.5	[-9.5, 2.5]	0.3489	no
	D2	59	53	6	82	-23.5	[-30.0, -17.0]	<0.0001	**yes**
	D3	97	15	18	70	+1.5	[-4.0, 7.5]	0.7283	no
	D4	98	14	19	69	+2.5	[-3.0, 8.0]	0.4869	no
	D5	2	110	0	88	-55.0	[-61.5, -48.0]	<0.0001	**yes**
GPT-6 Luna, no reasoning	D1	77	31	20	72	-5.5	[-12.5, 1.5]	0.1608	no
	D2	48	60	16	76	-22.0	[-30.0, -14.0]	<0.0001	**yes**
	D3	85	23	34	58	+5.5	[-2.0, 13.0]	0.1849	no
	D4	86	22	26	66	+2.0	[-5.0, 9.0]	0.6655	no
	D5	0	108	0	92	-54.0	[-61.0, -47.0]	<0.0001	**yes**
Claude Haiku 5.5, reasoning	D1	79	22	17	82	-2.5	[-8.5, 3.5]	0.5224	no
	D2	59	42	14	85	-14.0	[-20.5, -7.0]	0.0002	**yes**
	D3	84	17	27	72	+5.0	[-1.0, 11.5]	0.1742	no
	D4	94	7	29	70	+11.0	[5.5, 17.0]	0.0003	**yes**
	D5	0	101	0	99	-50.5	[-57.5, -44.0]	<0.0001	**yes**
Claude Haiku 5.5, no reasoning	D1	69	25	15	91	-5.0	[-11.0, 1.5]	0.1539	no
	D2	48	46	7	99	-19.5	[-26.0, -12.5]	<0.0001	**yes**
	D3	76	18	19	87	+0.5	[-5.5, 6.5]	1.0000	no
	D4	74	20	19	87	-0.5	[-6.5, 5.5]	1.0000	no
	D5	0	94	0	106	-47.0	[-54.5, -40.0]	<0.0001	**yes**
Qwen3-30B-A3B, no reasoning	D1	78	31	19	72	-6.0	[-13.0, 0.5]	0.1189	no
	D2	46	63	11	80	-26.0	[-34.0, -18.5]	<0.0001	**yes**
	D3	85	24	20	71	-2.0	[-8.5, 4.5]	0.6516	no
	D4	74	35	12	79	-11.5	[-18.5, -5.0]	0.0011	**yes**
	D5	0	109	0	91	-54.5	[-61.5, -48.0]	<0.0001	**yes**
Gemma 4 26B-A4B, no reasoning	D1	130	14	8	48	-3.0	[-7.5, 1.5]	0.2863	no
	D2	73	71	5	51	-33.0	[-40.5, -26.0]	<0.0001	**yes**
	D3	136	8	7	49	-0.5	[-4.5, 3.5]	1.0000	no
	D4	141	3	8	48	+2.5	[-0.5, 5.5]	0.2266	no
	D5	0	144	0	56	-72.0	[-78.5, -66.0]	<0.0001	**yes**
Gemma 4 26B-A4B, reasoning	D1	133	19	8	40	-5.5	[-10.5, -0.5]	0.0522	no
	D2	77	75	8	40	-33.5	[-41.5, -26.0]	<0.0001	**yes**
	D3	146	6	10	38	+2.0	[-1.5, 6.0]	0.4545	no
	D4	148	4	4	44	+0.0	[-2.5, 2.5]	1.0000	no
	D5	5	147	0	48	-73.5	[-79.5, -67.0]	<0.0001	**yes**

Haiku's D4 gain. Haiku with reasoning passes 29 tasks only in D4 against 7 only in D0 ($p = 0.0003$). The gain is absent with reasoning off (-0.5 pp), shrinks to $+5.0$ pp ($p = 0.13$) against the repeated D0 run, and is diffuse: the 29 tasks span 16 backend combinations, 19 involve the six parameters of Table 3 and 10 do not, and their D0 failures begin with a wrong argument in 12 cases, a never-called tool in 7 and no identifiable divergence in 10. We offer no mechanism for it.

Table 5: Run-to-run noise: a repeated D0 run of Claude Haiku 5.5 against the first, and each condition against both D0 runs. Same 200 tasks, same settings, eight hours apart.

Comparison	both pass	first only	second only	both fail	Δ pp	95% CI	McNemar p
D0 run 1 vs D0 run 2	92	9	21	78	+6.0	[1.0, 11.5]	0.0428
D0 run 1 vs D1	79	22	17	82	-2.5	[-8.5, 3.5]	0.5224
D0 run 2 vs D1	86	27	10	77	-8.5	[-14.5, -3.0]	0.0076
D0 run 1 vs D2	59	42	14	85	-14.0	[-20.5, -7.0]	0.0002
D0 run 2 vs D2	61	52	12	75	-20.0	[-27.0, -13.0]	<0.0001
D0 run 1 vs D3	84	17	27	72	+5.0	[-1.0, 11.5]	0.1742
D0 run 2 vs D3	97	16	14	73	-1.0	[-6.0, 4.5]	0.8555
D0 run 1 vs D4	94	7	29	70	+11.0	[5.5, 17.0]	0.0003
D0 run 2 vs D4	100	13	23	64	+5.0	[-1.0, 11.0]	0.1325
D0 run 1 vs D5	0	101	0	99	-50.5	[-57.5, -44.0]	<0.0001
D0 run 2 vs D5	0	113	0	87	-56.5	[-63.0, -49.5]	<0.0001
D0 run 1 vs D6	87	14	21	78	+3.5	[-2.0, 9.5]	0.3105
D0 run 2 vs D6	95	18	13	74	-2.5	[-8.0, 3.0]	0.4731
D0 run 1 vs D7	85	16	25	74	+4.5	[-2.0, 11.0]	0.2110
D0 run 2 vs D7	96	17	14	73	-1.5	[-7.0, 3.5]	0.7201

Table 6: Reasoning tokens per task by condition, mean over the 200 tasks.

Model	D0	D1	D2	D3	D4	D5
GPT-6 Luna, reasoning	330	368	390	328	335	2,294
GPT-6 Luna, no reasoning	0	0	0	0	0	0
Claude Haiku 5.5, reasoning	517	548	559	550	550	1,258
Claude Haiku 5.5, no reasoning	0	0	0	0	0	0
Qwen3-30B-A3B, no reasoning	0	0	0	0	0	0
Gemma 4 26B-A4B, no reasoning	0	0	0	0	2	0
Gemma 4 26B-A4B, reasoning	1,110	1,521	2,554	1,328	1,316	8,659

Table 7: Documentation sensitivity score $S_m = \frac{1}{N}\sum_i (P_{m,0} - P_{m,i})/P_{m,0}$, the mean relative drop from D0 over D1–D4 and over D1–D5. Because it averages a large D2 loss with near-zero changes elsewhere, it understates every model's dependence on parameter descriptions and is reported for completeness only.

Model	S (D1–D4)	S (D1–D5)
GPT-6 Luna, reasoning	0.103	0.279
GPT-6 Luna, no reasoning	0.093	0.274
Claude Haiku 5.5, reasoning	0.002	0.202
Claude Haiku 5.5, no reasoning	0.130	0.304
Qwen3-30B-A3B, no reasoning	0.209	0.367
Gemma 4 26B-A4B, no reasoning	0.118	0.294
Gemma 4 26B-A4B, reasoning	0.122	0.291

Table 8: Tool selection versus argument construction (%). "Sel." is tool-selection accuracy, "Arg." argument accuracy given the right tool, "1st tool" the share of tasks whose first call used a ground-truth tool, "1st exact" the share whose first call matched a ground-truth call exactly.

	D0				D2				D5			
Model	Sel.	Arg.	1st tool	1st exact	Sel.	Arg.	1st tool	1st exact	Sel.	Arg.	1st tool	1st exact
GPT-6 Luna, reasoning	89.2	94.4	56.5	55.0	82.8	83.3	59.5	44.0	43.3	60.9	8.5	3.0
GPT-6 Luna, no reasoning	91.8	91.4	58.5	56.0	88.1	79.5	58.0	40.0	45.4	36.1	8.5	2.0
Claude Haiku 5.5, reasoning	91.5	93.6	60.0	58.5	88.0	86.3	64.5	46.0	15.8	53.0	11.5	2.5
Claude Haiku 5.5, no reasoning	86.1	93.4	63.0	60.5	80.4	82.6	67.0	48.5	16.2	43.8	9.5	4.0
Qwen3-30B-A3B, no reasoning	91.9	90.7	62.0	58.5	89.2	79.7	65.5	46.0	54.6	21.3	6.5	2.5
Gemma 4 26B-A4B, no reasoning	96.8	93.2	63.5	62.0	87.9	82.1	65.0	47.5	22.5	23.3	21.5	4.0
Gemma 4 26B-A4B, reasoning	97.1	94.7	59.5	58.0	90.6	83.6	59.5	42.5	59.5	42.7	27.5	5.5

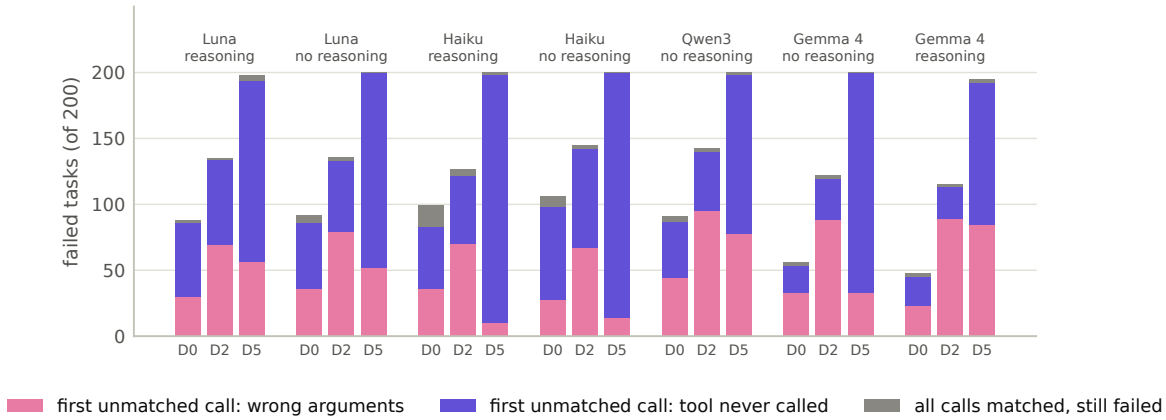


Figure 6: Earliest divergence in failed tasks: the first ground-truth call without an exact match, classified by whether the tool was called with wrong arguments (construction failure) or never called (selection failure); "all calls matched" means every ground-truth call was matched but the task still failed, usually through extra state-changing calls or a call made in a later turn than required.

Table 9: Earliest divergence among failed tasks (counts). "W.A.": wrong arguments; "N.C.": tool never called; "None": every ground-truth call matched but the task failed.

	D0			D2			D5		
Model	W.A.	N.C.	None	W.A.	N.C.	None	W.A.	N.C.	None
GPT-6 Luna, reasoning	30	56	2	69	65	1	56	138	4
GPT-6 Luna, no reasoning	36	50	6	79	54	3	52	148	0
Claude Haiku 5.5, reasoning	36	47	16	70	52	5	10	188	2
Claude Haiku 5.5, no reasoning	28	70	8	67	75	3	14	186	0
Qwen3-30B-A3B, no reasoning	44	43	4	95	45	3	78	120	2
Gemma 4 26B-A4B, no reasoning	33	20	3	88	31	3	33	167	0
Gemma 4 26B-A4B, reasoning	23	22	3	89	24	2	85	107	3

Table 10: Success (%) on the 91 tasks whose ground truth uses at least one of the six parameters of Table 3 ("six") and on the other 109 ("other"). The first column counts the tasks that pass in D0 and fail in D2 and, of those, how many are six-parameter tasks. D6 and D7 exist for each model's default reasoning setting.

	D0-only tasks	D0		D2		D6		D7	
Model	six / all	six	other	six	other	six	other	six	other
GPT-6 Luna, reasoning	44/53	51.6	59.6	4.4	56.0	50.5	56.9	49.5	54.1
GPT-6 Luna, no reasoning	50/60	59.3	49.5	6.6	53.2	–	–	–	–
Claude Haiku 5.5, reasoning	37/42	46.2	54.1	9.9	58.7	49.5	57.8	51.6	57.8
Claude Haiku 5.5, no reasoning	29/46	34.1	57.8	3.3	47.7	–	–	–	–
Qwen3-30B-A3B, no reasoning	50/63	60.4	49.5	8.8	45.0	59.3	44.0	49.5	40.4
Gemma 4 26B-A4B, no reasoning	64/71	75.8	68.8	5.5	67.0	75.8	65.1	64.8	64.2
Gemma 4 26B-A4B, reasoning	68/75	81.3	71.6	7.7	71.6	–	–	–	–

Table 11: D6 and D7 against D0 and against D2 on the same 200 tasks (each model in its default reasoning setting): success of the base condition and of D6 or D7, the discordant cells, the paired difference with its 95% bootstrap interval, and the exact McNemar p.

Model	Comparison	base %	D6/D7 %	base only	D6/D7 only	Δ pp	95% CI	p
GPT-6 Luna, reasoning	D6 vs D0	56.0	54.0	19	15	-2.0	[-7.5, 4.0]	0.6076
	D6 vs D2	32.5	54.0	6	49	+21.5	[15.0, 28.0]	<0.0001
	D7 vs D0	56.0	52.0	23	15	-4.0	[-10.0, 2.0]	0.2559
	D7 vs D2	32.5	52.0	7	46	+19.5	[13.0, 26.0]	<0.0001
Claude Haiku 5.5, reasoning	D6 vs D0	50.5	54.0	14	21	+3.5	[-2.0, 9.5]	0.3105
	D6 vs D2	36.5	54.0	7	42	+17.5	[11.0, 24.0]	<0.0001
	D7 vs D0	50.5	55.0	16	25	+4.5	[-2.0, 11.0]	0.2110
	D7 vs D2	36.5	55.0	13	50	+18.5	[11.0, 25.5]	<0.0001
Qwen3-30B-A3B, no reasoning	D6 vs D0	54.5	51.0	23	16	-3.5	[-10.0, 2.5]	0.3368
	D6 vs D2	28.5	51.0	6	51	+22.5	[16.0, 29.5]	<0.0001
	D7 vs D0	54.5	44.5	32	12	-10.0	[-16.5, -3.5]	0.0037
	D7 vs D2	28.5	44.5	14	46	+16.0	[9.0, 23.5]	<0.0001
Gemma 4 26B-A4B, no reasoning	D6 vs D0	72.0	70.0	13	9	-2.0	[-6.5, 2.5]	0.5235
	D6 vs D2	39.0	70.0	9	71	+31.0	[23.5, 39.0]	<0.0001
	D7 vs D0	72.0	64.5	25	10	-7.5	[-13.0, -2.0]	0.0167
	D7 vs D2	39.0	64.5	7	58	+25.5	[18.5, 32.5]	<0.0001

B Tool-use behaviour on BFCL

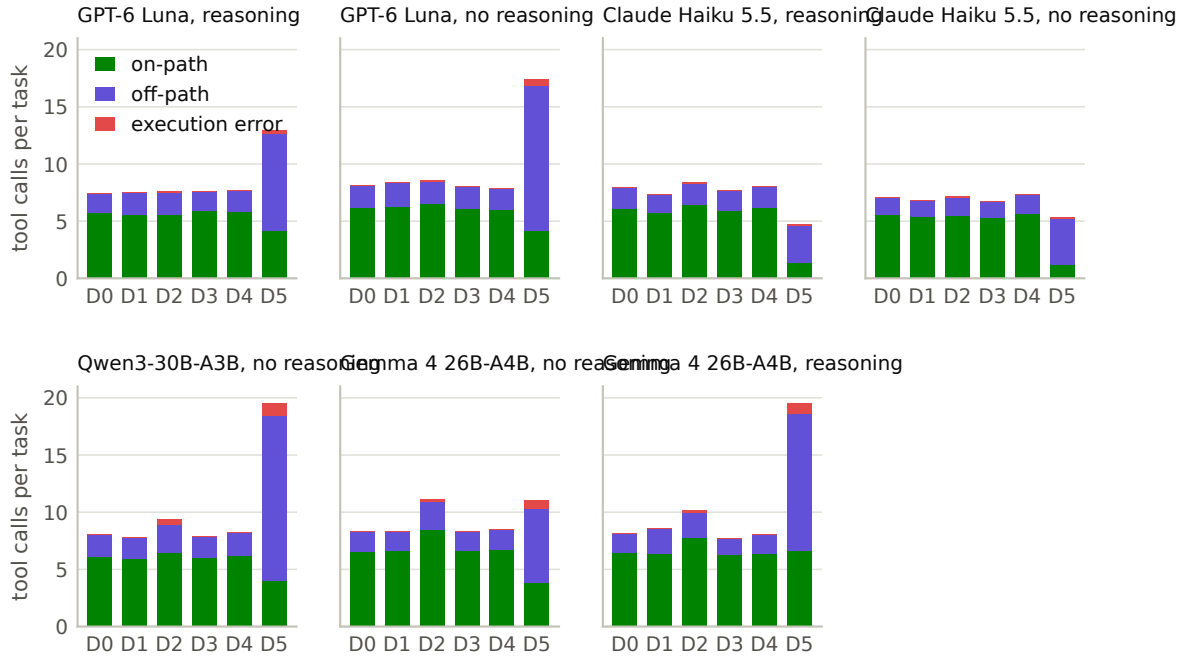


Figure 7: Composition of tool calls per task (mean over all 200 tasks, failures included): on-path calls, off-path calls that executed without error, and calls that returned an execution error.

D5 behaviour in detail. Haiku makes fewer calls than in D0 (4.7 against 7.9 with reasoning, 5.3 against 7.1 without), makes no call in the first turn of 34.5% of tasks (18.0% without reasoning), and most of its calls are off-path. Qwen3 makes 19.5 calls per task over 15.6 distinct tools of the roughly 28 available, with an execution error in 51% of tasks. Luna with reasoning sits between (12.9 calls, no first-turn call in 22% of tasks), as does Gemma 4 without reasoning (11.0 calls); the other setting of each probes instead (Luna without reasoning 17.4 calls, Gemma 4 with reasoning 19.5). BFCL's terminal errors follow the strategy: Haiku's and Luna's D5 failures with reasoning are dominated by turns that ended without a tool call (78 and 50 tasks), Qwen3's by the 20-step limit (57) and Gemma 4's by a wrong final state after probing (94); without reasoning Luna probes to a wrong final state (118) and Haiku splits (104 wrong states, 44 silent turns), and Gemma 4 with reasoning hits the step limit in 90 tasks. In D2 the extra failures are almost all wrong final states (for example 35 → 76 for Gemma 4): the model acted, but on the wrong values.

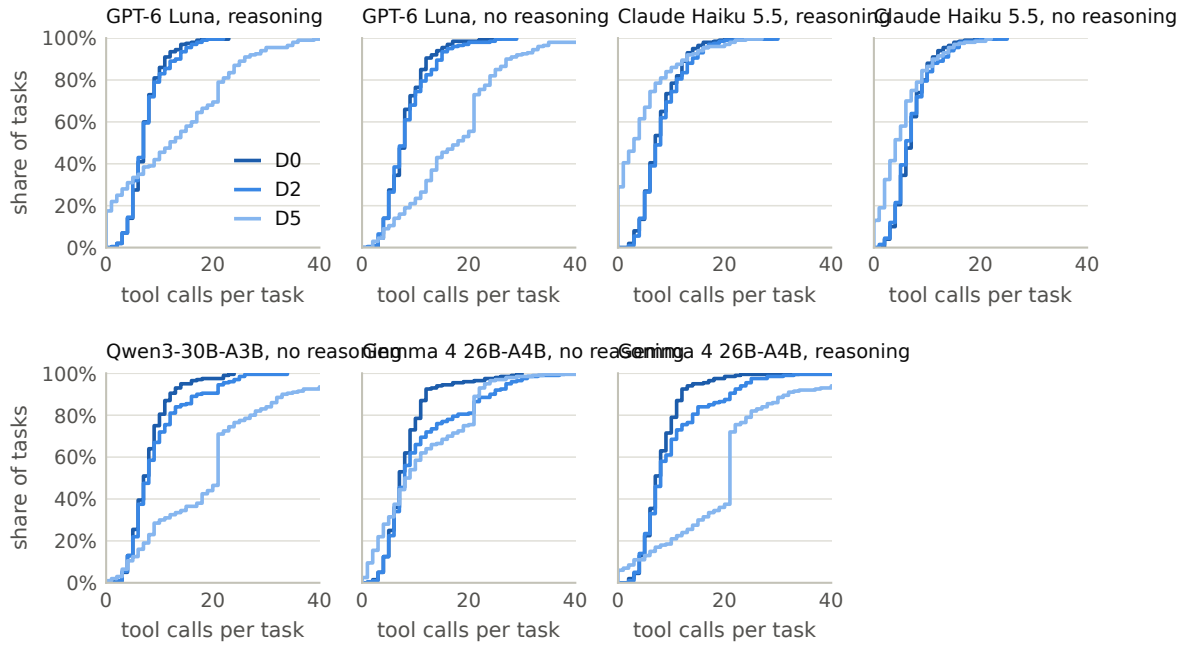


Figure 8: Empirical cumulative distribution of tool calls per task for D0, D2 and D5, all 200 tasks per curve. D2 shifts the distribution only slightly; D5 pulls it in opposite directions by model, Haiku rising steeply at 0 to 3 calls and Qwen3 stretching to the step cap.

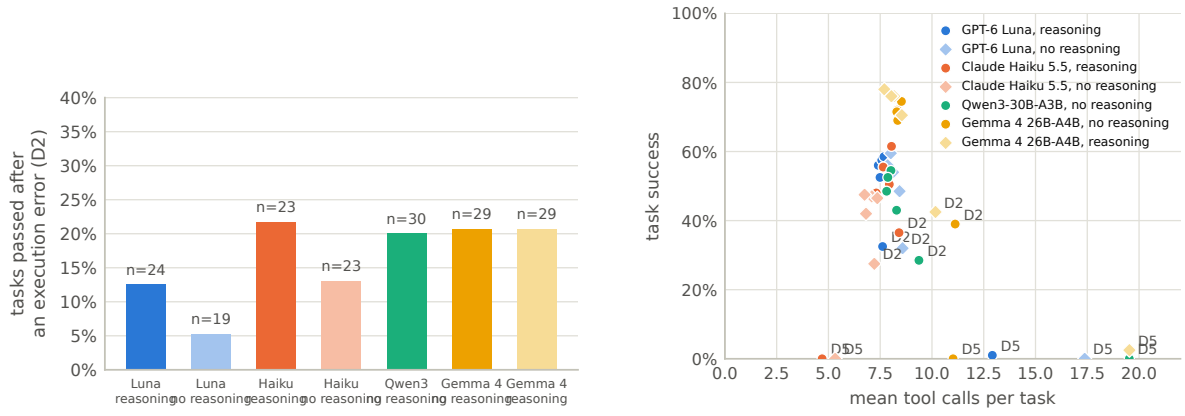


Figure 9: Left: recovery after an execution error in D2: share of tasks containing at least one failed call that still passed, with the number of such tasks. D0 denominators are 1 to 4 tasks and not comparable. **Right:** task success against mean tool calls per task, one point per configuration and condition. D0 through D4 sit at 7 to 9 calls per task; D5 moves points right (Qwen3, Luna, Gemma 4) or left (Haiku) without moving them up.

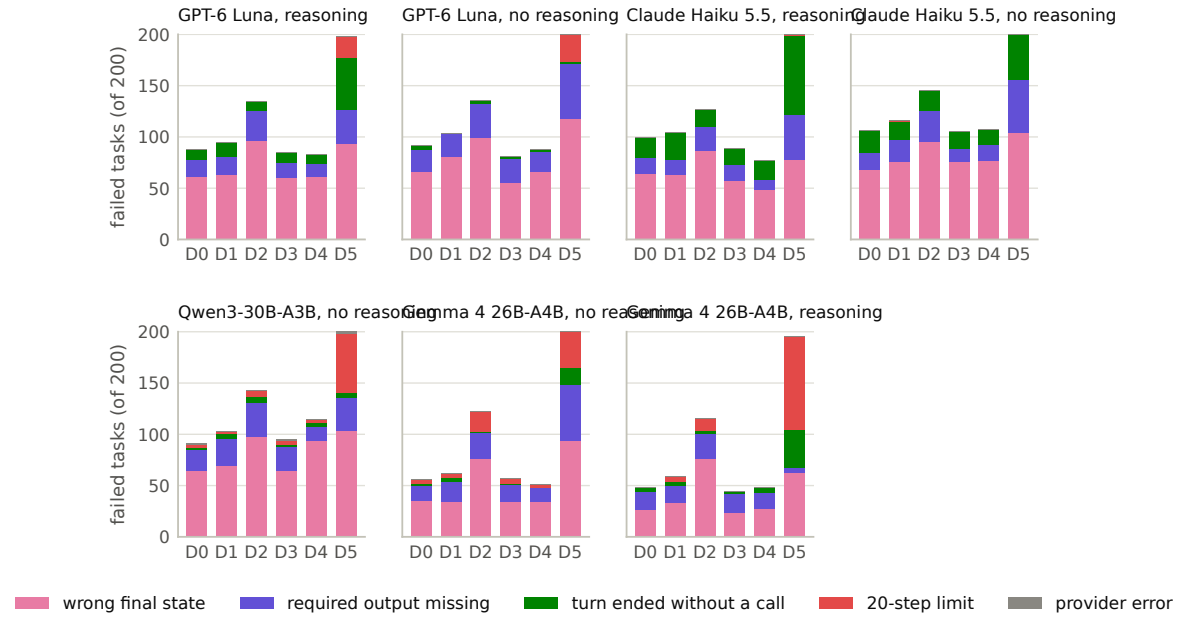


Figure 10: BFCL's terminal error per failed task, by condition. These are outcomes, not causes: a wrong final state can come from a wrong tool, a wrong argument or a missed step.

Table 12: Tool-use behaviour, mean per task over all 200 tasks (failures included). A repeated call has identical tool and arguments to an earlier call in the same task; "calls/pass" is the mean over passing tasks only.

Model	Cond.	calls	steps	on-path	off-path	failed	repeated	distinct tools	calls/pass
GPT-6 Luna, reasoning	D0	7.4	10.2	5.8	1.6	0.01	0.50	6.2	7.4
	D1	7.5	10.2	5.6	1.9	0.01	0.58	6.0	7.2
	D2	7.6	10.6	5.6	1.9	0.15	0.52	6.0	7.8
	D3	7.6	10.4	5.9	1.7	0.00	0.65	6.2	7.6
	D4	7.7	10.5	5.8	1.8	0.01	0.59	6.3	7.5
	D5	12.9	14.7	4.2	8.4	0.28	0.86	9.1	22.5
GPT-6 Luna, no reasoning	D0	8.1	11.5	6.2	1.9	0.03	0.86	6.4	7.9
	D1	8.4	11.8	6.2	2.1	0.07	0.92	6.6	7.8
	D2	8.6	12.1	6.5	1.9	0.10	0.93	6.3	8.0
	D3	8.0	11.4	6.1	1.9	0.01	0.89	6.5	7.6
	D4	7.9	11.2	6.0	1.9	0.01	0.69	6.4	7.7
	D5	17.4	20.3	4.2	12.6	0.56	2.50	12.1	–
Claude Haiku 5.5, reasoning	D0	7.9	10.1	6.1	1.8	0.01	0.70	6.3	7.6
	D1	7.3	9.7	5.7	1.6	0.00	0.52	6.0	6.9
	D2	8.4	10.6	6.4	1.8	0.12	0.85	6.2	7.8
	D3	7.6	10.0	6.0	1.7	0.01	0.58	6.2	7.4
	D4	8.0	10.2	6.2	1.9	0.01	0.78	6.4	8.0
	D5	4.7	7.3	1.4	3.2	0.10	0.25	3.5	–
Claude Haiku 5.5, no reasoning	D0	7.1	9.5	5.5	1.6	0.01	0.47	5.9	6.9
	D1	6.8	9.2	5.4	1.4	0.01	0.46	5.6	6.9
	D2	7.2	9.6	5.5	1.6	0.14	0.52	5.7	7.4
	D3	6.7	9.2	5.3	1.4	0.01	0.39	5.7	6.9
	D4	7.4	9.6	5.6	1.7	0.01	0.62	6.0	7.0
	D5	5.3	6.6	1.1	4.1	0.10	0.30	4.3	–
Qwen3-30B-A3B, no reasoning	D0	8.0	11.7	6.2	1.9	0.02	0.85	6.5	7.4
	D1	7.8	11.3	5.9	1.8	0.07	0.81	6.3	6.8
	D2	9.4	12.9	6.5	2.4	0.51	1.38	6.6	7.1
	D3	7.9	11.4	6.0	1.8	0.07	0.86	6.4	7.1
	D4	8.3	11.9	6.2	2.1	0.04	1.14	6.4	6.4
	D5	19.5	21.1	4.0	14.4	1.09	3.24	15.6	–
Gemma 4 26B-A4B, no reasoning	D0	8.3	11.4	6.6	1.8	0.02	1.24	6.3	7.4
	D1	8.3	11.5	6.7	1.7	0.00	1.29	6.2	7.4
	D2	11.1	13.8	8.5	2.4	0.23	3.38	5.9	7.2
	D3	8.3	11.4	6.6	1.6	0.02	1.27	6.3	7.4
	D4	8.5	11.6	6.8	1.8	0.01	1.32	6.2	7.8
	D5	11.0	14.0	3.9	6.4	0.73	3.94	5.0	–
Gemma 4 26B-A4B, reasoning	D0	8.2	11.4	6.4	1.7	0.00	0.90	6.4	8.0
	D1	8.5	11.6	6.4	2.1	0.01	1.18	6.3	7.7
	D2	10.2	13.3	7.7	2.2	0.19	1.24	6.2	8.2
	D3	7.7	10.9	6.3	1.4	0.00	0.72	6.3	7.6
	D4	8.0	11.2	6.3	1.7	0.00	0.80	6.4	7.8
	D5	19.5	21.5	6.6	12.0	0.89	0.92	12.6	11.4

Table 13: Recovery after an execution error: share of tasks containing at least one failed call that still passed, with the number of such tasks in parentheses.

Model	D0	D1	D2	D3	D4	D5
GPT-6 Luna, reasoning	100% (1)	67% (3)	12% (24)	– (0)	33% (3)	0% (38)
GPT-6 Luna, no reasoning	0% (5)	38% (8)	5% (19)	50% (2)	50% (2)	0% (67)
Claude Haiku 5.5, reasoning	50% (2)	– (0)	22% (23)	33% (3)	100% (1)	0% (17)
Claude Haiku 5.5, no reasoning	50% (2)	0% (2)	13% (23)	0% (1)	0% (2)	0% (17)
Qwen3-30B-A3B, no reasoning	25% (4)	0% (4)	20% (30)	8% (12)	0% (5)	0% (102)
Gemma 4 26B-A4B, no reasoning	33% (3)	– (0)	21% (29)	50% (4)	0% (1)	0% (30)
Gemma 4 26B-A4B, reasoning	– (0)	100% (1)	21% (29)	– (0)	– (0)	2% (81)

Table 14: Terminal error assigned by BFCL per failed task. "State": wrong final state; "Output": required output missing; "Empty": turn ended without a tool call; "Limit": 20-step limit reached; "Provider": inference error.

Model	Cond.	State	Output	Empty	Limit	Provider	Total failed
GPT-6 Luna, reasoning	D0	61	17	10	0	0	88
	D1	63	18	14	0	0	95
	D2	96	30	9	0	0	135
	D3	60	15	10	0	0	85
	D4	61	13	9	0	0	83
	D5	93	34	50	21	0	198
GPT-6 Luna, no reasoning	D0	66	22	4	0	0	92
	D1	81	22	0	0	0	103
	D2	99	34	3	0	0	136
	D3	55	24	2	0	0	81
	D4	66	20	2	0	0	88
	D5	118	54	1	27	0	200
Claude Haiku 5.5, reasoning	D0	64	16	19	0	0	99
	D1	63	15	26	0	0	104
	D2	86	24	17	0	0	127
	D3	57	16	16	0	0	89
	D4	49	9	19	0	0	77
	D5	78	43	78	1	0	200
Claude Haiku 5.5, no reasoning	D0	68	17	21	0	0	106
	D1	76	21	18	1	0	116
	D2	95	31	19	0	0	145
	D3	76	13	16	0	0	105
	D4	77	15	15	0	0	107
	D5	104	52	44	0	0	200
Qwen3-30B-A3B, no reasoning	D0	64	21	2	3	1	91
	D1	69	27	5	2	0	103
	D2	97	34	6	6	0	143
	D3	64	24	2	4	1	95
	D4	94	14	3	3	0	114
	D5	103	33	5	57	2	200
Gemma 4 26B-A4B, no reasoning	D0	35	15	2	4	0	56
	D1	34	20	4	4	0	62
	D2	76	26	0	20	0	122
	D3	34	17	1	5	0	57
	D4	34	14	0	3	0	51
	D5	94	54	17	35	0	200
Gemma 4 26B-A4B, reasoning	D0	26	18	4	0	0	48
	D1	33	17	4	5	0	59
	D2	76	25	3	11	0	115
	D3	24	18	2	0	0	44
	D4	27	16	5	0	0	48
	D5	63	4	38	90	0	195

C ToolSandbox: setup and results

Setup. ToolSandbox's 1 032 scenarios are 129 base scenarios in eight variants that vary distraction tools and apply the benchmark's own augmentations; we take the 91 base scenarios that need none of the five tools backed by a paid web API, in the variant that offers every tool the scenario does not withhold (24 to 28 tools per scenario; 28 in 73 of the 91), so that tool selection is part of every task as on BFCL. The user simulator is reached through OpenRouter with the same seed in every condition. ToolSandbox executes each call as a Python statement named after the provider's tool-call id; two providers return ids that are not valid identifiers, which the unmodified harness turns into a syntax error on every call, so the harness rewrites such ids before execution. When a model emits arguments that are not valid JSON, ToolSandbox aborts the scenario; like its own runner we score it 0 (27 of 2 184 plays, all Qwen3's). We report the mean similarity and the share of scenarios at similarity 1.0, and test paired differences with a sign-flip permutation test on the similarity and McNemar's test on the binary outcome.

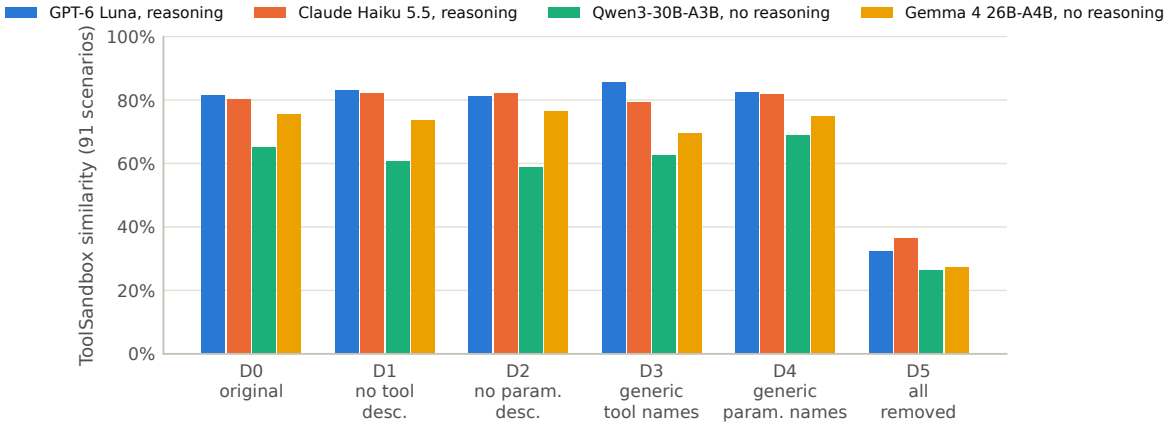


Figure 11: ToolSandbox mean similarity by condition (91 scenarios per bar). Removing any single component leaves every model within a few points of its baseline; removing all four costs 39 to 49 points.

Table 15: ToolSandbox results. Left: mean similarity (×100) over the 91 scenarios; right: share solved exactly (similarity 1.0).

	mean similarity						solved exactly (%)					
Model	D0	D1	D2	D3	D4	D5	D0	D1	D2	D3	D4	D5
GPT-6 Luna	81.5	82.9	81.2	85.4	82.5	32.3	25.3	31.9	25.3	31.9	29.7	14.3
Claude Haiku 5.5	80.1	82.1	82.2	79.3	81.8	36.3	26.4	29.7	25.3	25.3	29.7	17.6
Qwen3-30B-A3B	65.0	60.8	58.7	62.4	69.0	26.3	8.8	14.3	9.9	11.0	14.3	14.3
Gemma 4 26B-A4B	75.5	73.7	76.5	69.5	75.0	27.3	19.8	13.2	19.8	12.1	17.6	16.5

Results. Removing parameter descriptions (D2) changes mean similarity by −0.3, +2.0, −6.3 and +0.9 points for Luna, Haiku, Qwen3 and Gemma 4 (sign-flip $p \geq 0.07$; Table 16), and the share solved exactly is unchanged for every model (discordant cells 5/5, 2/1, 3/4, 2/2). D1 (+1.4, +2.0, −4.2, −1.8) and D4 are likewise within a few points, and so is D3 except for Gemma 4, which loses 6.0 points ($p = 0.003$) and seven exactly-solved scenarios against none gained ($p = 0.016$), the one single-component effect to reach nominal significance. D5 costs 39 to 49 points for every model ($p < 0.0001$).

The single-component nulls are what Section 6.2 predicts. Across the 72 parameters of the 28 tools an agent can see, none carries an allowed-value list the schema lacks; the only prose constraints are formats ("POSIX timestamp", "in 24hr format"), and most restate the parameter

name (`reminder_timestamp_lowerbound`, `hour`). Identifiers the tools exchange (`person_id`) are returned by earlier calls, so the agent copies rather than constructs them. The one suggestive exception points the same way: Qwen3's −6.3 is concentrated in the 36 *canonicalization* scenarios (70.0 to 58.5; Table 17), where the user states a date, duration or unit in surface form that the agent must convert before calling, that is, where a format convention has to be applied.

Tool selection survives every single-component condition. Taking as reference each model's D0 scenarios with similarity at least 0.9 in which it called a tool (54, 49, 53 and 41 for Haiku, Gemma 4, Luna and Qwen3), the share in which the ablated run calls every tool the D0 run used is 0.87 to 0.91 for Haiku under D1 to D4, 0.76 to 0.90 for Gemma 4, 0.74 to 0.83 for Luna and 0.66 to 0.76 for Qwen3, against 0.00 to 0.13 under D5. What D5 removes is the ability to tell 24 to 28 tools apart by anything but argument structure, and much of that is shared: six tools take no arguments, eight take one, and the four `set_*_status` tools all take a single boolean. The collapse looks as it does on BFCL: scenarios in which the agent never calls a tool rise from 6, 3, 7 and 0 (Luna, Haiku, Qwen3, Gemma 4) to 27, 19, 16 and 34; scenarios that run to the 30-message cap rise from 2 to 23 for Luna and 10 to 32 for Qwen3; two models mostly fall silent (Haiku, Gemma 4) and two probe to the limit (Luna, Qwen3). Because the user is simulated, its token usage rises by 40 to 180% in D5.

Two features of ToolSandbox limit what can be read from it. Its 18 *insufficient-information* scenarios, where the correct behaviour is to ask rather than act, score low for every model in every condition (24 to 74 at D0) and move by less than a point under D2. And the user is an LLM with the same seed in every condition but not deterministic, so each paired difference includes user noise; the sign-flip tests account for it but are not a substitute for a repeated run, which we did not make.

Table 16: ToolSandbox: every ablation against D0, paired over the 91 scenarios. Similarity columns are means (×100) with the bootstrap 95% CI of the paired difference and a sign-flip permutation p (10 000 permutations); exact-match columns give the share at similarity 1.0, the discordant cells (only in D0 / only in the ablation) and the exact McNemar p.

		mean similarity					solved exactly			
Model	Cond.	D0	abl.	Δ	95% CI	p	D0 %	abl. %	D0/abl. only	McNemar p
GPT-6 Luna	D1	81.5	82.9	+1.4	[-4.5, 7.4]	0.6518	25.3	31.9	5/11	0.2101
	D2	81.5	81.2	-0.3	[-5.7, 5.6]	0.9139	25.3	25.3	5/5	1.0000
	D3	81.5	85.4	+3.9	[-1.9, 9.9]	0.2012	25.3	31.9	5/11	0.2101
	D4	81.5	82.5	+1.0	[-5.0, 7.3]	0.7450	25.3	29.7	5/9	0.4240
	D5	81.5	32.3	-49.2	[-59.0, -39.4]	<0.0001	25.3	14.3	14/4	0.0309
Claude Haiku 5.5	D1	80.1	82.1	+2.0	[-3.4, 7.6]	0.4983	26.4	29.7	1/4	0.3750
	D2	80.1	82.2	+2.0	[-0.8, 5.4]	0.2035	26.4	25.3	2/1	1.0000
	D3	80.1	79.3	-0.8	[-5.0, 3.6]	0.7176	26.4	25.3	2/1	1.0000
	D4	80.1	81.8	+1.6	[-2.2, 6.5]	0.4793	26.4	29.7	1/4	0.3750
	D5	80.1	36.3	-43.8	[-54.8, -32.2]	<0.0001	26.4	17.6	16/8	0.1516
Qwen3-30B-A3B	D1	65.0	60.8	-4.2	[-13.3, 4.4]	0.3525	8.8	14.3	3/8	0.2266
	D2	65.0	58.7	-6.3	[-13.1, 0.4]	0.0706	8.8	9.9	3/4	1.0000
	D3	65.0	62.4	-2.5	[-9.6, 3.9]	0.4796	8.8	11.0	3/5	0.7266
	D4	65.0	69.0	+4.1	[-3.3, 11.7]	0.3038	8.8	14.3	3/8	0.2266
	D5	65.0	26.3	-38.7	[-51.1, -26.2]	<0.0001	8.8	14.3	5/10	0.3018
Gemma 4 26B-A4B	D1	75.5	73.7	-1.8	[-5.1, 1.3]	0.2700	19.8	13.2	6/0	0.0312
	D2	75.5	76.5	+0.9	[-2.9, 4.6]	0.6534	19.8	19.8	2/2	1.0000
	D3	75.5	69.5	-6.0	[-10.4, -2.3]	0.0029	19.8	12.1	7/0	0.0156
	D4	75.5	75.0	-0.5	[-5.8, 4.2]	0.8380	19.8	17.6	5/3	0.7266
	D5	75.5	27.3	-48.2	[-61.2, -34.5]	<0.0001	19.8	16.5	15/12	0.7011

Table 17: ToolSandbox mean similarity (×100) by scenario category, D0 against D2. Categories are ToolSandbox's own and overlap (n is the number of the 91 scenarios in each).

		GPT-6 Luna		Claude Haiku 5.5		Qwen3-30B-A3B		Gemma 4 26B-A4B	
Category	n	D0	D2	D0	D2	D0	D2	D0	D2
single tool call	13	90.2	89.8	92.8	92.7	84.6	79.9	91.3	91.4
multiple tool calls	60	81.9	81.4	86.3	89.3	72.3	64.5	87.6	89.0
single user turn	53	84.0	84.8	88.1	90.6	75.3	67.5	88.4	91.1
multiple user turns	20	81.6	77.8	85.5	88.1	72.3	66.7	87.9	85.0
state dependency	13	74.5	73.3	78.7	86.4	80.3	74.7	87.2	88.7
canonicalization	36	75.8	76.8	88.6	91.8	70.0	58.5	87.4	90.0
insufficient information	18	74.1	74.3	50.7	50.9	26.2	23.9	23.9	24.0

D Case studies

Cases were selected by one rule per model in its default reasoning setting: among tasks that pass in D0 and fail in D2, take the one with the fewest ground-truth calls (lowest id on ties), and show it in D0, D2 and D5. The rule favours short tasks so that the whole trajectory can be shown.

Claude Haiku 5.5, task 78 (Figure 1). Turn 0 is identical in D0 and D2. In turn 1, D0 passes the hashtag as `tags=["#BusinessOnTheMove"]`; D2 folds it into `content`, so the backend records a tweet with no tags. The only information that changed is the removed `tags` description. In D5 the model never calls a tool and says it will not guess which unlabeled tool posts to social media.

Gemma 4 26B-A4B, task 11. The user asks to list the files in a `/temp` directory (which does not exist) and tweet their names with `#fileshowcase`. D0 and D2 both wander in turn 0 and end it in the correct state; turn 1 is decided by the same hashtag-in-content error. In D5 the single call is `get_user_stats(username="ls -a /temp")`: a Twitter tool with the shell command as its argument.

GPT-6 Luna, task 177. The user books a flight from Los Angeles to Tokyo and asks to confirm the booking's details. D2 shows a recovered construction failure and an unrecovered selection failure: it first calls `get_flight_cost` with city names (the removed descriptions said airport codes), gets an error, corrects to `LAX` and `HND`, and books; in turn 1 D0 calls `retrieve_invoice` while D2 calls `get_booking_history`, and BFCL requires the invoice output. In D5 the model answers with an essay about verifying itineraries and never calls a tool.

Qwen3-30B-A3B, task 42. The user asks to create a file in an existing `Lectures` folder. In D0, after `cd Lectures` fails, the model calls `pwd`, finds it is already inside the folder, and proceeds. In D2 it stops after the failed `cd`, and in turn 1 creates a second `Lectures` folder inside the first, so the directory tree has an extra folder. Nothing about the documentation of the tools involved differs between D0 and D2; the run diverged on a near-tie decision and the evaluator is strict about state. In D5 the model tries eleven tools with guessed arguments and gives up.

Every decoded call and its result under D0, D2 and D5 follows. Results are truncated to about 100 characters; execution errors are in red.

GPT-6 Luna, reasoning, task `multi_turn_base_177`

Turn 0: Consider this: A patient, a traveler in this context, wants to book a flight using the card with id 'card_7629' and the access token 'abc123xyz', expiring on 10/2026, with CVV 456. They intend to fly from Los Angeles to Tokyo on December 15, 2026, in business class, estimated at $1,200.

Turn 1: Reflect on the importance of verifying information. Our traveler needs to confirm their flight booking's details for assurance.

Ground truth. t0: `book_flight(access_token='abc123xyz', card_id='card_7629', travel_date='2026-12-15', travel_from='LAX', travel_to='HND', travel_class='business');` t1: `retrieve_invoice(access_token='abc123xyz', booking_id='3426812')`

	D0 original **pass** (6 steps, 5 calls)	D2 no parameter descriptions **fail**, required output missing (6 steps, 6 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)

t0	`get_nearest_airport_by_city({"location": "Los Angeles"})`
	`-> {"nearest_airport": "LAX"}`
	`get_nearest_airport_by_city({"location": "Tokyo"})`
	`-> {"nearest_airport": "HND"}`
	`get_flight_cost({"travel_from": "LAX", "travel_to": "HND", "travel_date": "2026-12-15", "travel_class": "business"})`
	`-> {"travel_cost_list": [860.0]}`
	`book_flight({"access_token": "abc123xyz", "card_id": "card_7629", "travel_date": "2026-12-15", "travel_from": "LAX", "travel_to": "HND", "travel_class": "busin...)`
	`-> {"booking_id": "3426812", "transaction_id": "45451592", "booking_status": true, "booking_history"...`
		`get_nearest_airport_by_city({"location": "Los Angeles"})`
		`-> {"nearest_airport": "LAX"}`
		`get_nearest_airport_by_city({"location": "Tokyo"})`
		`-> {"nearest_airport": "HND"}`
		`get_flight_cost({"travel_from": "Los Angeles", "travel_to": "Tokyo", "travel_date": "2026-12-15", "travel_class": "business"})`
		`-> Error during execution: No available route for the given airports.`
		`get_flight_cost({"travel_from": "LAX", "travel_to": "HND", "travel_date": "2026-12-15", "travel_class": "business"})`
		`-> {"travel_cost_list": [860.0]}`
		`book_flight({"access_token": "abc123xyz", "card_id": "card_7629", "travel_date": "2026-12-15", "travel_from": "LAX", "travel_to": "HND", "travel_class": "busin...)`
		`-> {"booking_id": "3426812", "transaction_id": "45451592", "booking_status": true, "booking_history"...`
			(no tool call)

t1	`retrieve_invoice({"access_token": "abc123xyz", "booking_id": "3426812"})` `-> {"invoice": {"booking_id": "3426812", "travel_date": "2026-12-15", "travel_from": "LAX", "travel_...`	`get_booking_history({"access_token": "abc123xyz"})` `-> {"booking_history": {"3426812": {"card_id": "card_7629", "travel_date": "2026-12-15", "travel_fro...`	*(no tool call)*

continued on next page

	D0 original **pass** (6 steps, 5 calls)	D2 no parameter descriptions **fail**, required output missing (6 steps, 6 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)
	Final reply: "I verified the booking invoice. It confirms: - **Route:** Los Angeles (LAX) to Tokyo Haneda (HND) - **Date:** December 15, 2026 - **Class:** Business - **Fare:** $860 - **Booking ID:** 3426812 - **Transaction ID:** 45451592"	*Final reply:* "I verified the booking against the travel system. Booking **3426812** is listed as: - **Route:** Los Angeles (LAX) → Tokyo Haneda (HND) - **Date:** December 15, 2026 - **Class:** Business - **Recorded fare:** $860 - **Card:** card_7629 - **Transaction ID:** 45451592 These details match the booking…"	*Final reply:* "Verifying the itinerary before relying on it helps prevent costly mistakes—especially with an international trip. Check the exact travel date, departure and arrival airports, times and time zones, business-class cabin, passenger name, baggage rules, and change or cancellation terms. Confirm that the…"

Claude Haiku 5.5, reasoning, task `multi_turn_base_78`

Turn 0: I'm gearing up for a quick business getaway and need my ride all set. Would you be able to verify if my tire pressure is in check? If it falls under 37.5 PSI, perhaps we could swing by the nearest tire shop?

Turn 1: While we head over to ensure my tires are roadworthy, I'd like to send out a swift update on my business account. Let's post a tweet: 'Ensuring my wheels are well-maintained. Maintenance is key to success!' with the hashtag 'BusinessOnTheMove'.

Ground truth. t0: `check_tire_pressure()`, `find_nearest_tire_shop()`; t1: `post_tweet(content='Ensuring my wheels are well-maintained. Maintenance is key to success!', tags=['#BusinessOnTheMove'])`

	D0 original **pass** (5 steps, 3 calls)	D2 no parameter descriptions **fail**, wrong final state (5 steps, 3 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)

t0	`check_tire_pressure({})`
	`-> {"frontLeftTirePressure": 35.0, "frontRightTirePressure": 35.0, "rearLeftTirePressure": 33.0, "re...`
	`find_nearest_tire_shop({})`
	`-> {"shopLocation": "456 Oakwood Avenue, Rivermist, 83214"}`
		`check_tire_pressure({})`
		`-> {"frontLeftTirePressure": 35.0, "frontRightTirePressure": 35.0, "rearLeftTirePressure": 33.0, "re...`
		`find_nearest_tire_shop({})`
		`-> {"shopLocation": "456 Oakwood Avenue, Rivermist, 83214"}`
			(no tool call)

t1	`post_tweet({"content": "Ensuring my wheels are well-maintained. Maintenance is key to success!", "tags": ["#BusinessOnTheMove"]})` `-> {"id": 10, "username": "businesspro", "content": "Ensuring my wheels are well-maintained. Mainten...`	`post_tweet({"content": "Ensuring my wheels are well-maintained. Maintenance is key to success! #BusinessOnTheMove"})` `-> {"id": 10, "username": "businesspro", "content": "Ensuring my wheels are well-maintained. Mainten...`	*(no tool call)*

continued on next page

	D0 original **pass** (5 steps, 3 calls)	D2 no parameter descriptions **fail**, wrong final state (5 steps, 3 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)
	Final reply: "Your tweet is posted on the account @businesspro: "Ensuring my wheels are well-maintained. Maintenance is key to success!" #BusinessOnTheMove I haven't set navigation to the tire shop yet. Do you want me to send you to 456 Oakwood Avenue, Rivermist, 83214?"	*Final reply:* "Your tweet is posted from the businesspro account: "Ensuring my wheels are well-maintained. Maintenance is key to success! #BusinessOnTheMove" Do you want me to set navigation to the tire shop at 456 Oakwood Avenue, Rivermist, 83214?"	*Final reply:* "I can't post that yet. None of my tools are labeled as social media or tweeting functions, so I'd be guessing which one to use, and posting is a public action I don't want to get wrong. Your tweet is ready to go: "Ensuring my wheels are well-maintained. Maintenance is key to success! #BusinessOnTh..."

Qwen3-30B-A3B, no reasoning, task `multi_turn_base_42`

Turn 0: In my existing folder called 'Lectures', I'd like to set up a new file called 'Notes2023.txt'.

Turn 1: Inside 'Notes2023.txt', I shall jot down some pivotal reminders: 'Study diligently, practice programming, master algorithms.'

Turn 2: Ascertain for me how many characters of text remain in this alphabetically sorted version of 'Notes2023.txt'.

Ground truth. t0: `touch(file_name='Notes2023.txt')`; t1: `echo(content='Study diligently, practice programming, master algorithms.', file_name='Notes2023.txt')`; t2: `wc(file_name='Notes2023.txt', mode='c')`

	D0 original **pass** (11 steps, 8 calls)	D2 no parameter descriptions **fail**, wrong final state (12 steps, 9 calls)	D5 all removed **fail**, wrong final state (14 steps, 11 calls)
t0	`cd({"folder": "Lectures"})` `-> {"error": "cd: 'Lectures': No such file or directory"}`	`cd({"folder": "Lectures"})` `-> {"error": "cd: 'Lectures': No such file or directory"}`	`cat({"file_name": "Lectures/Notes2023.txt"})` `-> {"error": "cat: 'Lectures/Notes2023.txt': Invalid character"}`
	`ls({})` `-> {"current_directory_content": []}`	`ls({})` `-> {"current_directory_content": []}`	`cd({"folder": "Lectures/Notes2023.txt"})` `-> {"error": "cd: Lectures/Notes2023.txt: Unsupported path. Only one folder level at a time is suppo...`
	`pwd({})` `-> {"current_working_directory": "/Lectures"}`		`cp({"source": "Lectures", "destination": "exists"})` `-> {"error": "cp: cannot copy 'Lectures': No such file or directory"}`
	`ls({})` `-> {"current_directory_content": []}`		`diff({"file_name1": "Lectures", "file_name2": "create"})` `-> {"error": "diff: Lectures or create: No such file or directory"}`

continued on next page

	D0 original **pass** (11 steps, 8 calls)	D2 no parameter descriptions **fail**, wrong final state (12 steps, 9 calls)	D5 all removed **fail**, wrong final state (14 steps, 11 calls)

	`touch({"file_name": "Notes2023.txt"})`
	`-> None`
			`du({})`
			`-> {"disk_usage": "0 bytes"}`
			`echo({"content": "Lectures"})`
			`-> {"terminal_output": "Lectures"}`
			`find({"path": "Lectures/Notes2023.txt"})`
			`-> {"error": "find: 'Lectures/Notes2023.txt': No such file or directory"}`
			`grep({"file_name": "Lectures", "pattern": "list"})`
			`-> {"error": "grep: Lectures: No such file or directory"}`
			`ls({})`
			`-> {"current_directory_content": []}`
			`mkdir({"dir_name": "Lectures"})`
			`-> None`
			`mv({"source": "Notes2023.txt", "destination": "create"})`
			`-> {"error": "mv: cannot move 'Notes2023.txt': No such file or directory"}`

t1	`echo({"content": "Study diligently, practice programming, master algorithms.", "file_name": "Notes2023.txt"})`
	`-> None`
		`mkdir({"dir_name": "Lectures"})`
		`-> None`
		`cd({"folder": "Lectures"})`
		`-> {"current_working_directory": "Lectures"}`
		`echo({"content": "Study diligently, practice programming, master algorithms.", "file_name": "Notes2023.txt"})`
		`-> {"error": "echo: cannot write to 'Notes2023.txt': No such file"}`
		`touch({"file_name": "Notes2023.txt"})`
		`-> None`
			(no tool call)

continued on next page

	D0 original **pass** (11 steps, 8 calls)	D2 no parameter descriptions **fail**, wrong final state (12 steps, 9 calls)	D5 all removed **fail**, wrong final state (14 steps, 11 calls)
		`echo({"content": "Study diligently, practice programming, master algorithms.", "file_name": "Notes2023.txt"})` `-> None`	

t2	`sort({"file_name": "Notes2023.txt"})`
	`-> {"sorted_content": "Study diligently, practice programming, master algorithms. "}`
	`wc({"file_name": "Notes2023.txt", "mode": "c"})`
	`-> {"count": 58, "type": "characters"}`
		`sort({"file_name": "Notes2023.txt"})`
		`-> {"sorted_content": "Study diligently, practice programming, master algorithms. "}`
		`wc({"file_name": "Notes2023.txt", "mode": "c"})`
		`-> {"count": 58, "type": "characters"}`
			(no tool call)

	Final reply: "The alphabetically sorted version of 'Notes2023.txt' contains 58 characters. This includes all letters, spaces, and punctuation marks in the text "Study diligently, practice programming, master algorithms.""	*Final reply:* "I've analyzed the 'Notes2023.txt' file and found the following: The file contains 58 characters in total (including spaces and punctuation). When sorted alphabetically, the content remains the same: "Study diligently, practice programming, master algorithms." This is because the text is already in..."	*Final reply:* "It seems that the file 'Notes2023.txt' could not be created or accessed due to system limitations. As a result, there is no content to analyze or count characters from. If you'd like, I can help you draft the text you want to include in the file, or guide you through creating it in a different envi..."

Gemma 4 26B-A4B, no reasoning, task `multi_turn_base_11`

Turn 0: Display all the available files located within the '/temp' directory including hidden one using the terminal interface .

Turn 1: With the information presented on the terminal, draft a post on Twitter mentioning only the file names seperated them by comma and space, and include the hashtag '#fileshowcase' to spread the word.

Ground truth. t0: `ls(a=True)`; t1: `post_tweet(content='file1.txt, file2.txt', tags=['#fileshowcase'])`

	D0 original **pass** (15 steps, 13 calls)	D2 no parameter descriptions **fail**, wrong final state (13 steps, 11 calls)	D5 all removed **fail**, required output missing (3 steps, 1 calls)
t0	`cd({"folder": "temp"})` `-> {"error": "cd: 'temp': No such file or directory"}`	`ls({"a": true})` `-> {"current_directory_content": ["file1.txt", "file2.txt"]}`	`get_user_stats({"username": "ls -a /temp"})` `-> {"tweet_count": 0, "following_count": 0, "retweet_count": 0}`

continued on next page

	D0 original **pass** (15 steps, 13 calls)	D2 no parameter descriptions **fail**, wrong final state (13 steps, 11 calls)	D5 all removed **fail**, required output missing (3 steps, 1 calls)
	`ls({"a": true})` `-> {"current_directory_content": ["file1.txt", "file2.txt"]}`	`cd({"folder": "/temp"})` `-> {"error": "cd: /temp: Unsupported path. Only one folder level at a time is supported."}`	
	`find({"path": "/temp"})` `-> {"error": "find: '/temp': No such file or directory"}`	`ls({"a": true})` `-> {"current_directory_content": ["file1.txt", "file2.txt"]}`	
	`ls({"a": true})` `-> {"current_directory_content": ["file1.txt", "file2.txt"]}`	`find({"path": "/temp"})` `-> {"error": "find: '/temp': No such file or directory"}`	
	`find({"path": "/"})` `-> {"matches": ["/file1.txt", "/file2.txt"]}`	`ls({"a": true})` `-> {"current_directory_content": ["file1.txt", "file2.txt"]}`	
	`ls({"a": true})` `-> {"current_directory_content": ["file1.txt", "file2.txt"]}`	`find({"path": "."})` `-> {"matches": ["./file1.txt", "./file2.txt"]}`	
	`find({"path": "/"})` `-> {"matches": ["/file1.txt", "/file2.txt"]}`	`ls({"a": true})` `-> {"current_directory_content": ["file1.txt", "file2.txt"]}`	
	`find({"path": "/"})` `-> {"matches": ["/file1.txt", "/file2.txt"]}`	`find({"path": "/"})` `-> {"matches": ["/file1.txt", "/file2.txt"]}`	
	`find({"path": "/"})` `-> {"matches": ["/file1.txt", "/file2.txt"]}`	`find({"path": "/"})` `-> {"matches": ["/file1.txt", "/file2.txt"]}`	
	`find({"path": "/"})` `-> {"matches": ["/file1.txt", "/file2.txt"]}`	`find({"path": "/temp"})` `-> {"error": "find: '/temp': No such file or directory"}`	
	`find({"name": "temp", "path": "/"})` `-> {"matches": []}`		
	`ls({"a": true})` `-> {"current_directory_content": ["file1.txt", "file2.txt"]}`		
t1	`post_tweet({"content": "file1.txt, file2.txt", "tags": ["#fileshowcase"]})` `-> {"id": 1, "username": "michael", "content": "file1.txt, file2.txt", "tags": ["#fileshowcase"], "m...`	`post_tweet({"content": "file1.txt, file2.txt #fileshowcase"})` `-> {"id": 1, "username": "michael", "content": "file1.txt, file2.txt #fileshowcase", "tags": [], "me...`	*(no tool call)*

continued on next page

	D0 original **pass** (15 steps, 13 calls)	D2 no parameter descriptions **fail**, wrong final state (13 steps, 11 calls)	D5 all removed **fail**, required output missing (3 steps, 1 calls)
	Final reply: “I have posted the tweet for you: "file1.txt, file2.txt #fileshowcase".”	*Final reply:* “The tweet has been posted successfully: "file1.txt, file2.txt #fileshowcase"”	*Final reply:* “Since no files were found in the ‘/temp‘ directory, I cannot list any file names. If you can provide the file names or if they appear after a different command, I will be happy to draft the tweet for you!”